# The Capacitated Vehicle Routing Problem (CVRP)

CSE 462: Algorithm Engineering Sessional
Final Presentation

**Team Members**

| | |
|---|---|
| Sabbir Alam Saad | (ID: 2005059) |
| Tanvir Hossain | (ID: 2005014) |
| Md. Afzal Hossain Tanim | (ID: 2005008) |
| Reduanul Islam Imon | (ID: 2005040) |
| Suhaeb Bin Matin | (ID: 2005086) |
| Mohammad Ali Bhuiyan | (ID: 2005113) |

Department of Computer Science and Engineering (CSE)
Bangladesh University of Engineering and Technology (BUET)

*30th March 2026*

# What is the Capacitated Vehicle Routing Problem?

**Definition**

The Capacitated Vehicle Routing Problem (CVRP) is a combinatorial optimization problem where a fleet of vehicles with limited capacity must deliver goods to multiple customers from a central depot, minimizing total travel distance or cost [1].

| Central Depot | Fleet of Vehicles | Limited Capacity | Customer Demands |
|---|---|---|---|
| Starting and ending point for all vehicles | Multiple vehicles used to serve all customers | Each vehicle has maximum load capacity | Each customer has specific delivery requirements |

# Key Components of CVRP

- **Depot (Central Warehouse):** A single starting location where all vehicles begin and end their routes
- **Vehicle Fleet:** A set of $m$ identical or homogeneous vehicles, each with capacity $Q$
- **Customers:** A set of $n$ customers, each located at specific coordinates with demand $d_i$
- **Distance Matrix:** Known distances or costs $c_{ij}$ between all pairs of locations (depot and customers)
- **Capacity Constraint:** Total demand on any route cannot exceed vehicle capacity $Q$

# CVRP Mathematical Formulation

**Objective Function**

$$\text{Minimize: } \sum_i \sum_j c_{ij} \cdot x_{ij}$$

(Total travel cost / distance)

**Constraints**

1. **Visit each customer once:**

$$\sum_j x_{ij} = 1 \quad \forall i$$

2. **Capacity constraint:**

$$\sum_i d_i \leq Q$$

3. **Flow conservation:**
   Flow in = Flow out
4. **Depot condition:**
   Start and end at depot

# CVRP Example

**Problem Setup:**

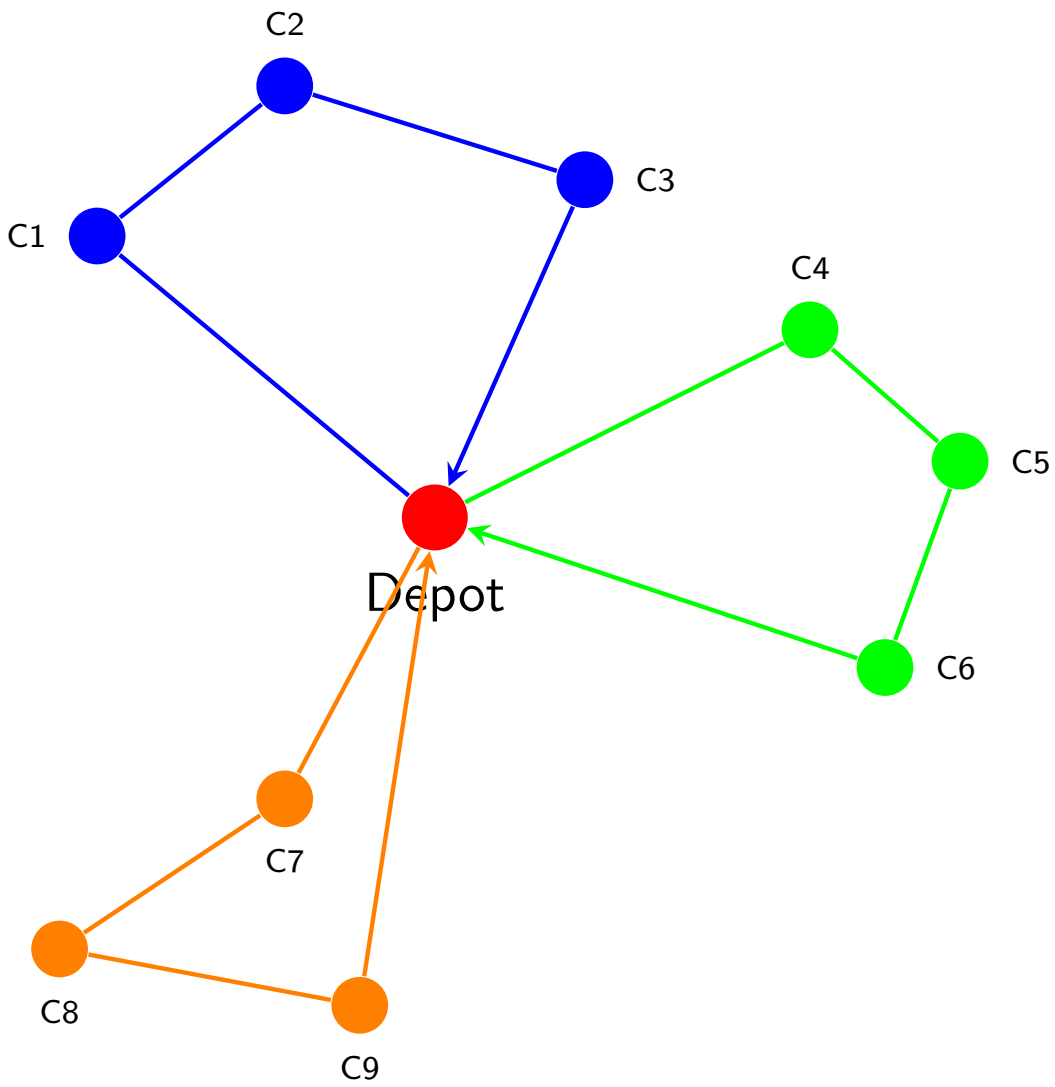
- Depot: Central warehouse
- Vehicles: 3 trucks
- Capacity: 100 units each
- Customers: 9 locations

**Customer Demands:**

| | | |
|---|---|---|
| C1: 30 | C2: 25 | C3: 40 |
| C4: 35 | C5: 45 | C6: 20 |
| C7: 50 | C8: 30 | C9: 15 |

**Solution:**

- Route 1: 95 units
- Route 2: 100 units
- Route 3: 95 units



# Proving CVRP is NP-Hard

**Our Approach**

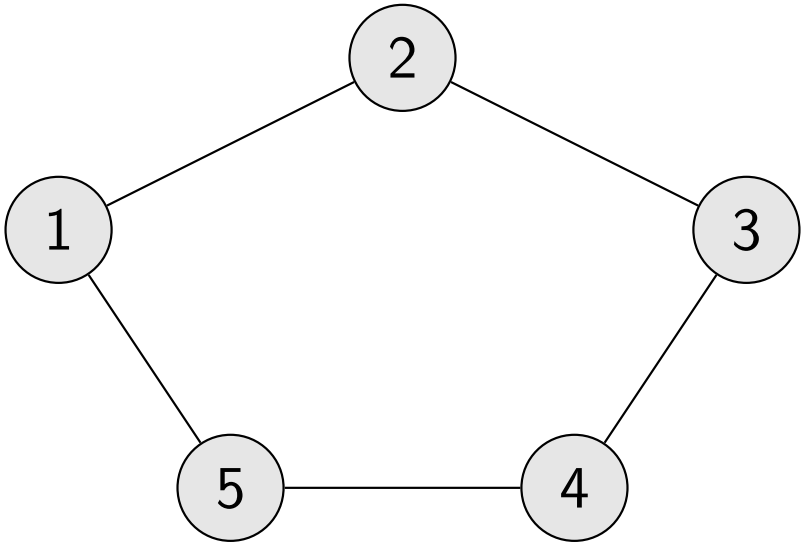
We prove CVRP is NP-Hard by showing a polynomial-time reduction from the Traveling Salesman Problem (TSP).

**Known:**
TSP is NP-Hard

**To Prove:**
CVRP is NP-Hard

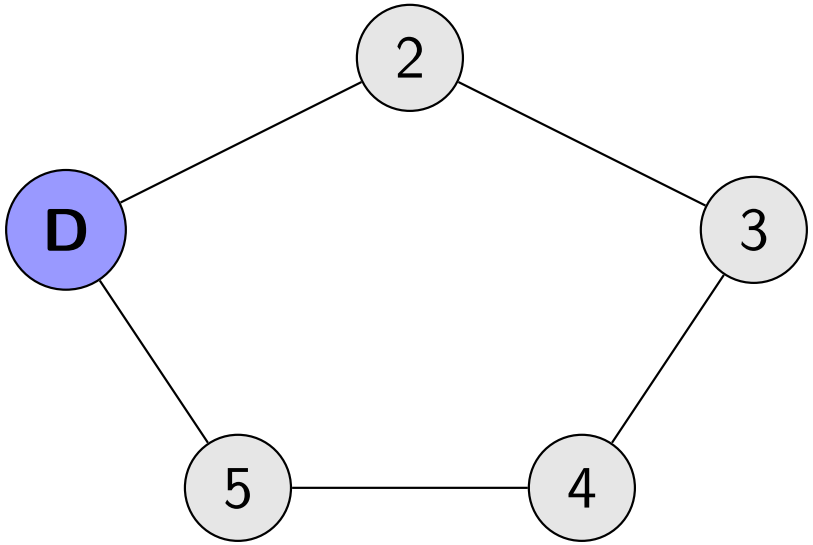
$$\text{TSP} \leq_P \text{CVRP}$$

# Step 1: Traveling Salesman Problem (TSP)



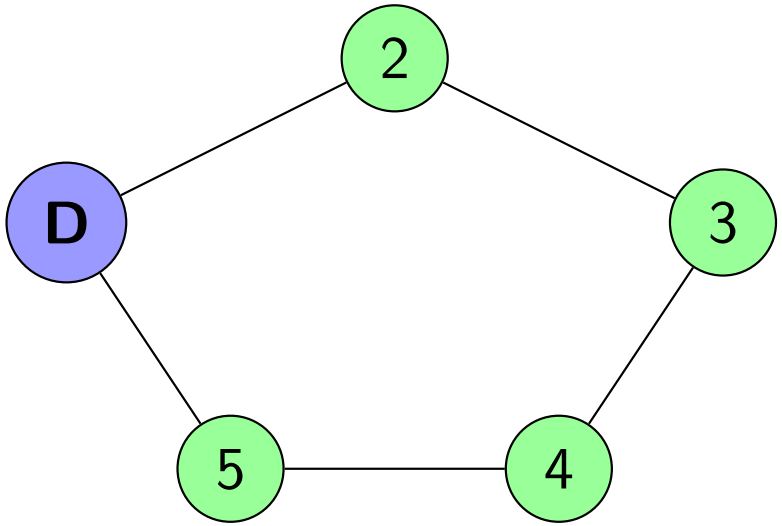

All nodes are identical — find a minimum Hamiltonian cycle

# Step 2: Choose a Depot




Select any node as the depot

# Building the CVRP Instance (from TSP)

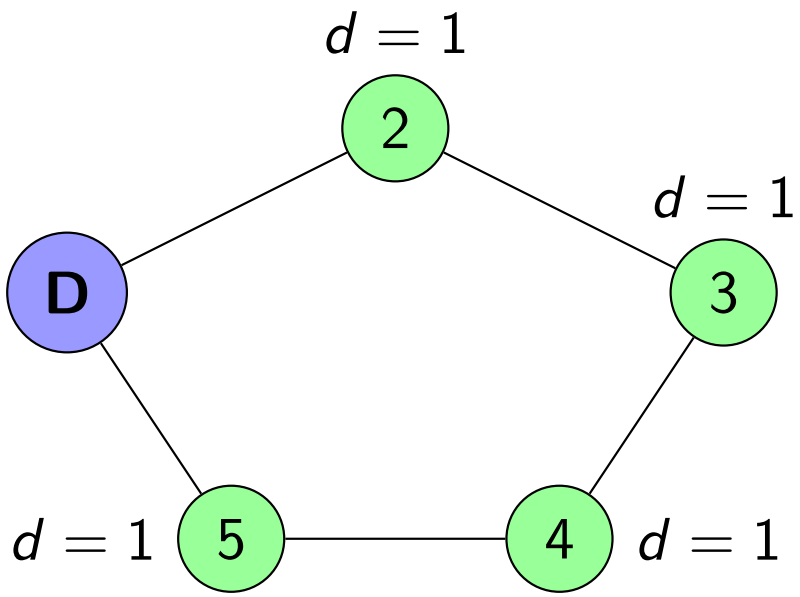



Define customers

**CVRP Parameters**

Customers: n - 1

# Building the CVRP Instance (from TSP)

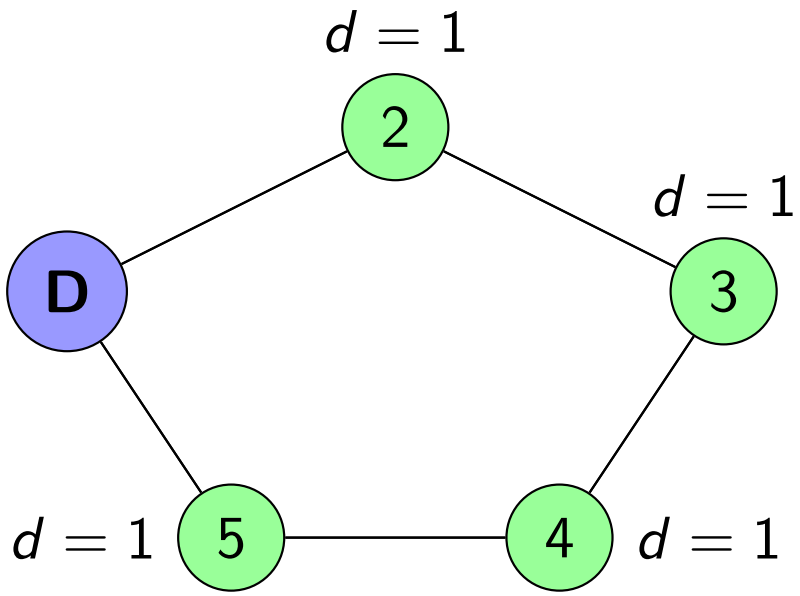



Assign unit demands

**CVRP Parameters**

Customers: n - 1
Demand: $d_i = 1$

# Building the CVRP Instance (from TSP)

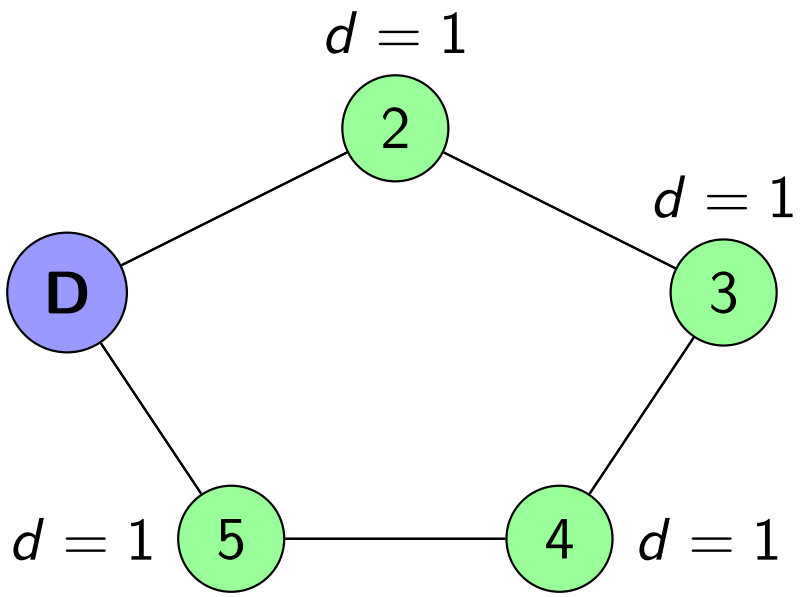



Restrict to one vehicle

**CVRP Parameters**

Customers: n - 1
Demand: $d_i = 1$
Vehicles: $m = 1$

# Building the CVRP Instance (from TSP)




Capacity forces a single tour

**CVRP Parameters**

Customers: n - 1
Demand: $d_i = 1$
Vehicles: $m = 1$
Capacity: $Q = n - 1$

# Key Insight

## Crucial Observation

By setting

$$vehicles = 1, \quad demand = 1, \quad capacity = n - 1$$

## Equivalence

One CVRP route visiting all customers $\Longleftrightarrow$ One TSP Hamiltonian cycle

# NP-Hardness Conclusion

$$\text{TSP} \leq_p \text{CVRP}$$

**Conclusion**

Since TSP is NP-Hard, the Capacitated Vehicle Routing Problem is NP-Hard.

# Algorithm Categories Defined I

Before detailing specific CVRP algorithms, we must define the classes of algorithms used in operations research.

1. **Exact Algorithms**
   - **Definition:** Algorithms that guarantee finding the *globally optimal* solution.
   - **Pros:** Proof of optimality.
   - **Cons:** Exponential time complexity ($O(2^n)$ or worse). Only feasible for small instances ($n < 200$).

# Algorithm Categories Defined II

2. **Approximation Algorithms**
   - **Definition:** Polynomial-time algorithms that provide a guarantee on the solution quality.
   - **The Guarantee ($\alpha$):** An algorithm is an $\alpha$-approximation if for any instance, the solution $Sol \leq \alpha \times Opt$.
   - **Context:** For general CVRP, finding a constant factor approximation is difficult due to the hardness of Bin Packing.

# Algorithm Categories Defined III

3. **Heuristics**
   - **Definition:** "Rule of thumb" techniques. They use problem-specific knowledge to find "good enough" solutions quickly.
   - **No Guarantees:** Unlike approximation algorithms, they do not guarantee a bound on the error.
   - **Types:**
     - *Constructive:* Build a solution from scratch.
     - *Improvement:* Take an existing solution and try to make it better.

# Algorithm Categories Defined IV

**4. Metaheuristics**

- **Definition:** High-level problem-independent frameworks that guide local search heuristics to escape local optima.
- **Mechanism:** They balance *Exploration* (searching new areas of the solution space) and *Exploitation* (refining the current best solution).
- **Examples:** Genetic Algorithms, Tabu Search, Simulated Annealing.

# Existing Algorithms

| Category | Algorithm | Key Features |
|---|---|---|
| **Exact** | Branch and Bound [2] | Enumerates candidate solutions by forming a rooted tree |
| | **Branch-and-Cut** [1] | Uses capacity cuts and comb inequalities. |
| | Branch-Cut-and-Price [3] | State-of-the-art; uses Column Generation. |
| **Approximation** | Christofides' Ext. [4] | Extensions of TSP approx; mostly theoretical. |
| **Heuristic** | Clarke & Wright [5] | Greedy merging based on "savings" ($s_{ij}$). |
| | **Sweep Algorithm** [6] | Clusters by polar angle, solves TSP per cluster. |
| **Metaheuristic** | Tabu Search [7] | Local search with memory structures. |
| | **Genetic Algorithm** [8] | Population-based; often uses "Giant Tour" split. |
| | Large Neighborhood Search [9] | Destroy part of the solution and recreate it |

# Branch and Bound (B&B)

**Concept:** Systematically enumerates candidate solutions by forming a rooted tree [2].

**Mechanism:**

1. **Branching:** Splits the problem into smaller subproblems (e.g., "Edge $x_{12}$ must be used" vs "Edge $x_{12}$ cannot be used").
2. **Bounding:** Calculates a lower bound for each subproblem (usually via Linear Relaxation).
3. **Pruning:** If a subproblem's Lower Bound $>$ Current Best Solution, that branch is discarded (pruned).

# Branch and Cut (B&C)

**Enhancement to B&B:** Standard B&B is weak because the Linear Relaxation of CVRP is often too loose. B&C adds *Valid Inequalities* (Cuts) to tighten the relaxation [1].

**Famous Cuts:**

- **Rounded Capacity Cuts:** Ensures demand constraints are met for subsets.
- **Comb Inequalities:** Generalized from TSP to handle connectivity.
- **Framed Capacity Cuts:** Specific to VRP, combining capacity and geometric constraints.

*Status:* Currently effective for $n \approx 100 - 150$.

# Branch-Cut-and-Price (BCP)

**The State-of-the-Art:** Combines Branch-and-Bound with **Column Generation** [3].
**Set Partitioning Formulation:** Instead of deciding edges ($x_{ij}$), we decide which *Routes* ($r_k$) to select.

$$\min \sum_{r \in \Omega} c_r \lambda_r$$

Subject to: Each customer is in exactly one route.
**Column Generation:** Since the set of possible routes $\Omega$ is exponential, we start with a few. We generate new "promising" routes dynamically by solving a **Pricing Subproblem** (usually a Shortest Path Problem with Resource Constraints - SPPRC).
*Status:* Can solve instances up to $n \approx 200 - 300$.

# Constructive Heuristics

These algorithms build a feasible solution from scratch.

**A. Clarke and Wright Savings Algorithm** [5]

- **Init:** $n$ routes $(0 \to i \to 0)$.
- **Metric:** Savings $s_{ij} = c_{i0} + c_{0j} - c_{ij}$.
- **Step:** Merge routes with max $s_{ij}$ if capacity allows.
- **Complexity:** $O(n^2 \log n)$.

**B. Sweep Algorithm** [6]

- **Geometric approach.**
- Sort customers by polar angle around the depot.
- "Sweep" around, adding customers to a vehicle until capacity $Q$ is full, then start a new vehicle.
- Solve TSP for each cluster.

# Tabu Search (TS)

**Mechanism:** [7]

- Performs local search (like Swap/Relocate).
- **The Tabu List:** To prevent cycling ($A \to B \to A$), recent moves are declared "Tabu" (forbidden) for a certain number of iterations.
- **Aspiration Criterion:** Can override Tabu status if the move produces a new global best solution.

# Genetic Algorithms (GA)

**Biological Analogy:** Evolution [8].

- **Population:** A set of solutions (Chromosomes).
- **Crossover:** Combine parents to create offspring (e.g., Order Crossover).
- **Mutation:** Randomly tweak a child to maintain diversity.
- **Selection:** Survival of the fittest.

**VRP Specialization:** Most successful VRP GAs use a **"Giant Tour"** representation (a single sequence of all customers) and a **Split Algorithm** to optimally break the sequence into routes.

# Ant Colony Optimization (ACO)

**Biological Analogy:** Foraging ants [10].

- **Pheromones:** Artificial agents (ants) construct routes probabilistically based on "pheromone trails" $\tau_{ij}$ on edges.
- **Update:** Good routes receive more pheromone; unused edges evaporate.
- **Result:** Over time, the colony converges on the optimal path.

# Large Neighborhood Search (LNS)

**Mechanism:** Ruin and Recreate [9].

- **Ruin:** Intentionally destroy part of the solution (e.g., remove 20% of customers randomly or spatially).
- **Recreate:** Re-insert the removed customers using a greedy heuristic or regret heuristic.
- **ALNS (Adaptive LNS):** Automatically selects which "Ruin" and "Recreate" operators work best for the current instance.

# Exact Algorithm Implementation

Two independent Python implementations were developed to solve the CVRP exactly, both tested on standard `.vrp` benchmark instances.

| **Aspect** | **Custom B&C** | **OR-Tools** |
|---|---|---|
| LP / MIP engine | `scipy` HiGHS | OR-Tools CP engine |
| Cuts added | Capacity + Comb | Internal (hidden) |
| First solution | Nearest-neighbour + 2-opt | Multi-strategy |
| Improvement | B&B with LIFO stack | GLS / Tabu / SA |
| Optimality proof | Yes (within node budget) | Yes (time limit) |
| Scalability | $n \lesssim 50$ | $n \lesssim 200$ |
| Dependencies | `numpy, scipy` | `ortools` |

Both solvers accept standard TSPLIB `.vrp` files and are executed via the shell scripts `run_bc.sh` and `run_all.sh` for batch benchmarking.

# Results: Custom B&C vs. OR-Tools (1/2)

## Experimental Setup

Tested on **27 Augerat Set-A benchmarks** ($n = 31$–$79$ customers). **Time limits:** OR-Tools = 360 s, Custom B&C = 600 s. Known optimal distances used as ground truth.

| Instance | $n$ | $k$ | Optimal | B&C | B&C Gap (%) | OR-Tools | ORT Gap (%) |
|---|---|---|---|---|---|---|---|
| A-n32-k5 | 31 | 5 | 784 | 926 | 18.11 | 784 | 0.00 |
| A-n33-k5 | 32 | 5 | 661 | 661 | 0.00 | 661 | 0.00 |
| A-n33-k6 | 32 | 6 | 742 | 836 | 12.67 | 742 | 0.00 |
| A-n34-k5 | 33 | 5 | 778 | 805 | 3.47 | 778 | 0.00 |
| A-n36-k5 | 35 | 5 | 799 | 1031 | 29.04 | 799 | 0.00 |
| A-n37-k5 | 36 | 5 | 669 | 756 | 13.00 | 669 | 0.00 |
| A-n37-k6 | 36 | 6 | 949 | 1271 | 33.93 | 949 | 0.00 |
| A-n38-k5 | 37 | 5 | 730 | 986 | 35.07 | 730 | 0.00 |
| A-n39-k5 | 38 | 5 | 822 | 1028 | 25.06 | 825 | 0.36 |
| A-n39-k6 | 38 | 6 | 831 | 957 | 15.16 | 833 | 0.24 |
| A-n44-k6 | 43 | 6 | 937 | 1343 | 43.33 | 939 | 0.21 |
| A-n45-k6 | 44 | 6 | 944 | 1422 | 50.64 | 951 | 0.74 |
| A-n45-k7 | 44 | 7 | 1146 | 1383 | 20.68 | 1146 | 0.00 |

# Results: Custom B&C vs. OR-Tools (2/2)

| Instance | $n$ | $k$ | Optimal | B&C | B&C Gap (%) | OR-Tools | ORT Gap (%) |
|---|---|---|---|---|---|---|---|
| A-n48-k7 | 47 | 7 | 1073 | 1453 | 35.41 | 1073 | 0.00 |
| A-n53-k7 | 52 | 7 | 1010 | 1385 | 37.13 | 1017 | 0.69 |
| A-n54-k7 | 53 | 7 | 1167 | 1386 | 18.77 | 1170 | 0.26 |
| A-n55-k9 | 54 | 9 | 1073 | 1472 | 37.19 | 1074 | 0.09 |
| A-n60-k9 | 59 | 9 | 1354 | 1775 | 31.09 | 1354 | 0.00 |
| A-n62-k8 | 61 | 8 | 1288 | 1713 | 33.00 | 1306 | 1.40 |
| A-n63-k10 | 62 | 10 | 1314 | 1860 | 41.55 | 1321 | 0.53 |
| A-n63-k9 | 62 | 9 | 1616 | 2139 | 32.36 | 1641 | 1.55 |
| A-n64-k9 | 63 | 9 | 1401 | 1903 | 35.83 | 1419 | 1.28 |
| A-n65-k9 | 64 | 9 | 1174 | 1694 | 44.29 | 1184 | 0.85 |
| A-n69-k9 | 68 | 9 | 1159 | 1477 | 27.44 | 1170 | 0.95 |
| A-n80-k10 | 79 | 10 | 1763 | 2290 | 29.89 | 1791 | 1.59 |
| **Average** | | | | | **28.08** | | **1.06** |

**Custom B&C (limit: 600 s)**

- Avg. gap: **28.08%**
- Gap range: 0% – 50.64%

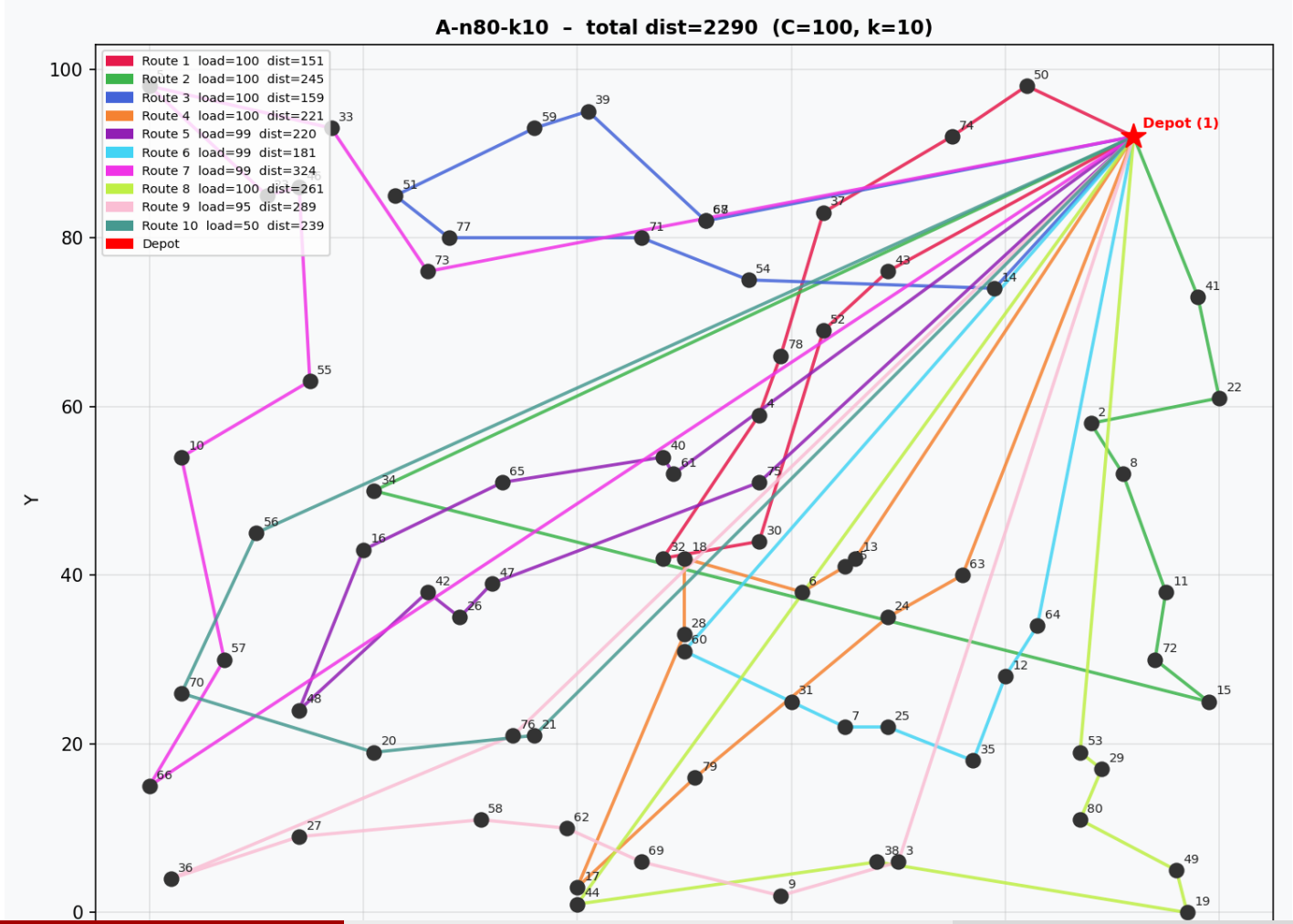
**OR-Tools (limit: 360 s)**

- Avg. gap: **1.06%**
- Gap range: 0% – 1.59%

# Custom B&C Solution — A-n80-k10

**Instance: A-n80-k10** $n = 79$, $k = 10$, Optimal $= 1763$

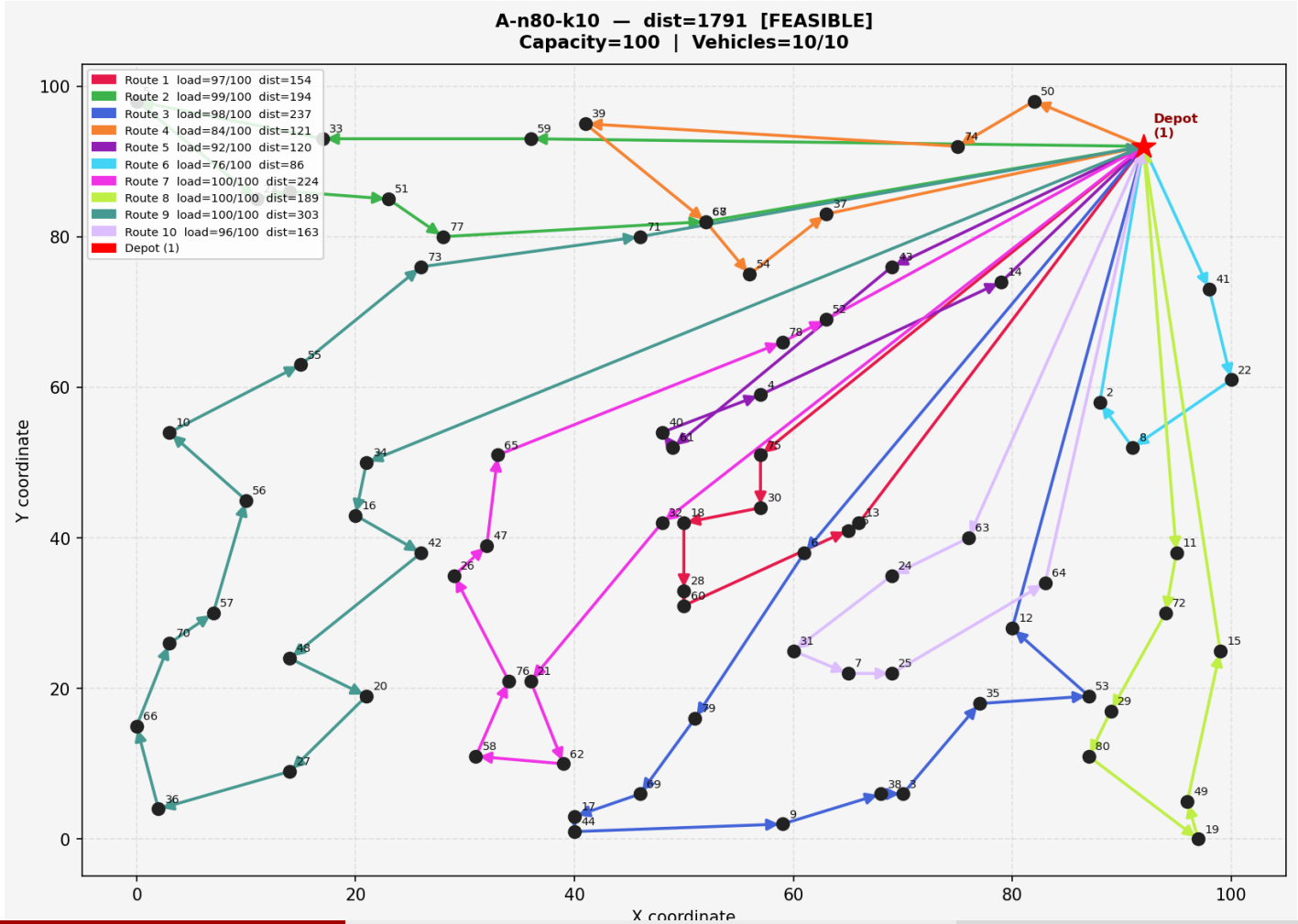
B&C result: **2290** Gap: **29.89%** Time limit: 600 s



# OR-Tools Solution — A-n80-k10

**Instance: A-n80-k10** $n = 79$, $k = 10$, Optimal $= 1763$

OR-Tools result: **1791** Gap: **1.59%** Time limit: 360 s



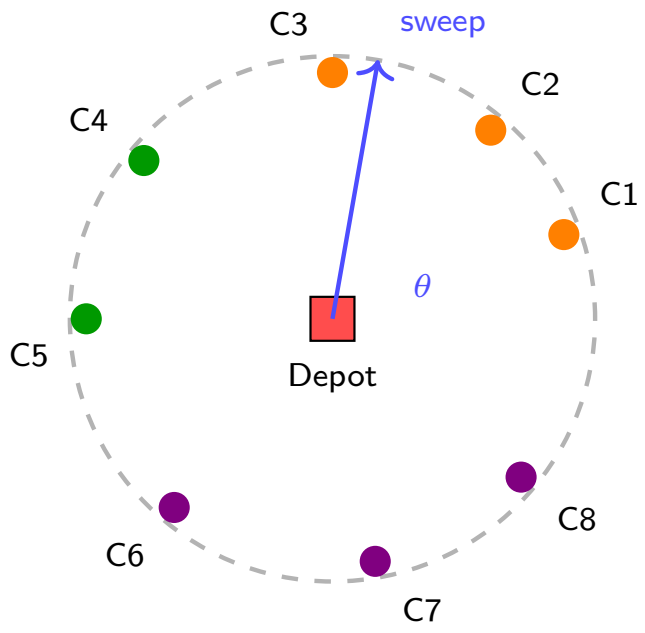
# Gillett & Miller Sweep — Algorithm Overview

## Core Idea

Customers are sorted by their polar angle around the depot. A "sweep" groups them into vehicle routes as the angle increases, respecting the capacity constraint $Q$. Each cluster is then optimised as a TSP sub-tour.

**Steps:**

1. Compute polar angle $\theta_i = \text{atan2}(y_i - y_0,\ x_i - x_0)$ for every customer $i$
2. Sort customers by $\theta_i \in [0, 2\pi)$
3. Greedily assign customers to the current vehicle until $\sum d_i > Q$; then open a new vehicle
4. For each cluster: construct a route via **nearest-neighbour** heuristic
5. Improve each route with **2-opt**

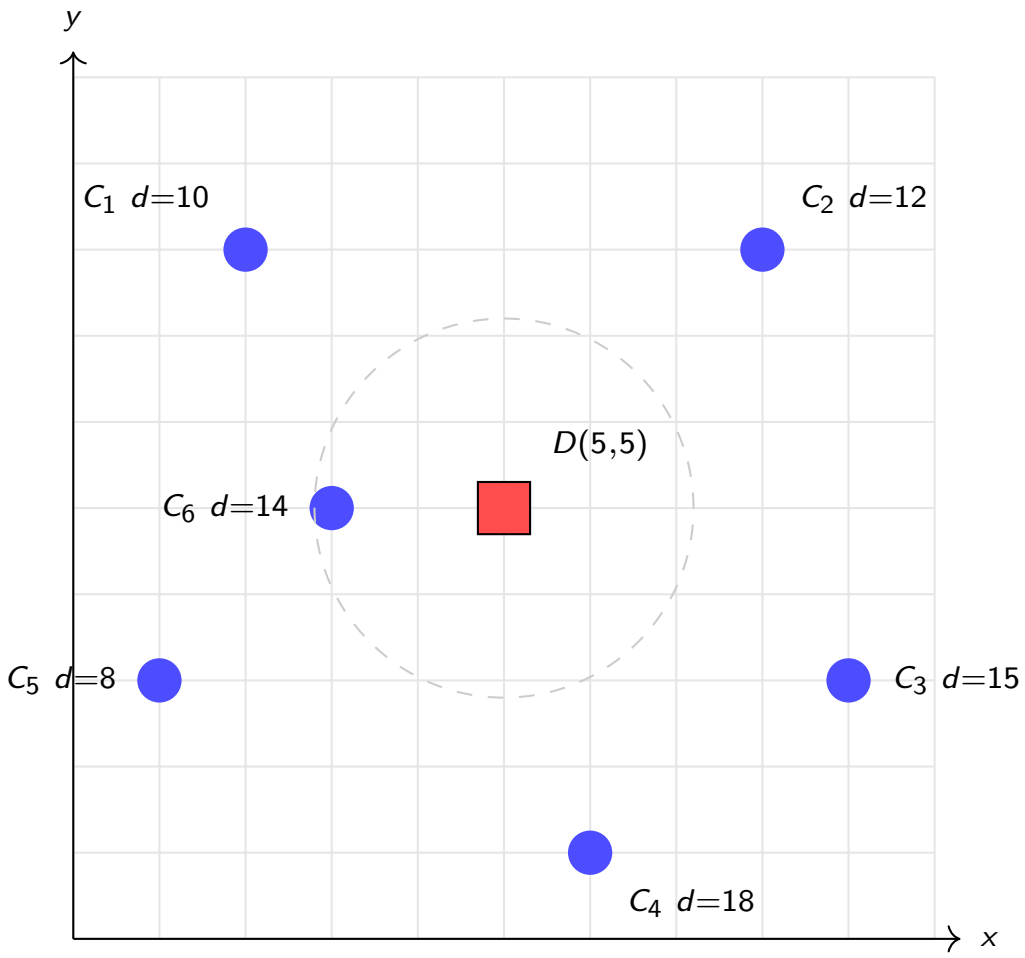


# Worked Example — Problem Instance

**Setup:**

- Depot $D$ at $(5, 5)$
- Vehicle capacity $Q = 30$
- 6 customers

| Node | $(x, y)$ | $d_i$ | $\theta_i$ |
|---|---|---|---|
| $D$ | $(5, 5)$ | — | — |
| $C_1$ | $(2, 8)$ | 10 | 135 |
| $C_2$ | $(8, 8)$ | 12 | 45 |
| $C_3$ | $(9, 3)$ | 15 | 333 |
| $C_4$ | $(6, 1)$ | 18 | 296 |
| $C_5$ | $(1, 3)$ | 8 | 207 |
| $C_6$ | $(3, 5)$ | 14 | 180 |



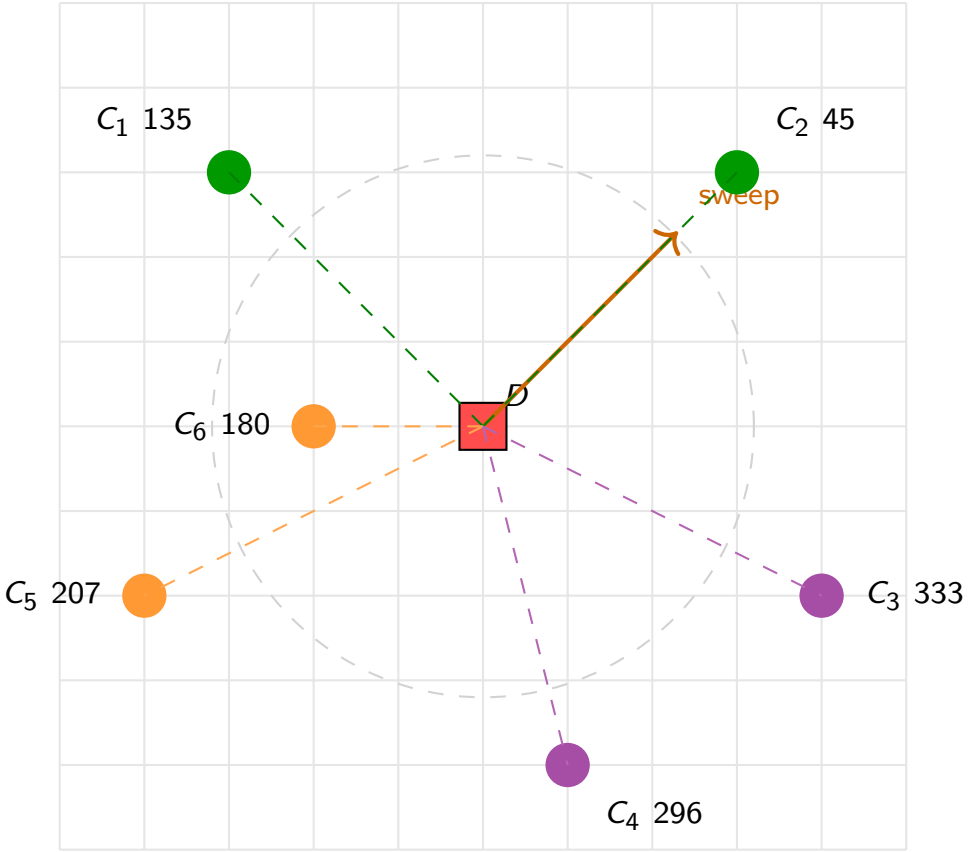
# Gillett Sweep — Step 1: Sort by Polar Angle

**Polar angle from depot $D = (5, 5)$**

$\theta_i = \text{atan2}(y_i - 5,\ x_i - 5)$, mapped to $[0, 360)$

**Sorted order:**

| Rank | Node | $\theta$ | $d_i$ |
|---|---|---|---|
| 1 | $C_2$ | 45 | 12 |
| 2 | $C_1$ | 135 | 10 |
| 3 | $C_6$ | 180 | 14 |
| 4 | $C_5$ | 207 | 8 |
| 5 | $C_4$ | 296 | 18 |
| 6 | $C_3$ | 333 | 15 |



# Gillett Sweep — Step 2: Greedy Clustering ($Q = 30$)

Process angle-sorted customers; close a route when adding the next customer would exceed $Q$:

| Step | Node | $d_i$ | Cum. load | Action | Route |
|---|---|---|---|---|---|
| 1 | $C_2$ | 12 | 12 | Add to V1 | V1: $\{C_2\}$ |
| 2 | $C_1$ | 10 | 22 | Add to V1 | V1: $\{C_2, C_1\}$ |
| 3 | $C_6$ | 14 | **36**>30 | Close V1, open V2 | V2: $\{C_6\}$ |
| 4 | $C_5$ | 8 | 22 | Add to V2 | V2: $\{C_6, C_5\}$ |
| 5 | $C_4$ | 18 | **40**>30 | Close V2, open V3 | V3: $\{C_4\}$ |
| 6 | $C_3$ | 15 | **33**>30 | Close V3, open V4 | V4: $\{C_3\}$ |

**4 clusters formed**

| R1 | R2 | R3 | R4 |
|---|---|---|---|
| $C_2, C_1$ | $C_6, C_5$ | $C_4$ | $C_3$ |
| load $= 22$ | load $= 22$ | load $= 18$ | load $= 15$ |

# Gillett Sweep — Step 3: Nearest-Neighbour Routes

**R1** cluster $\{C_2, C_1\}$:

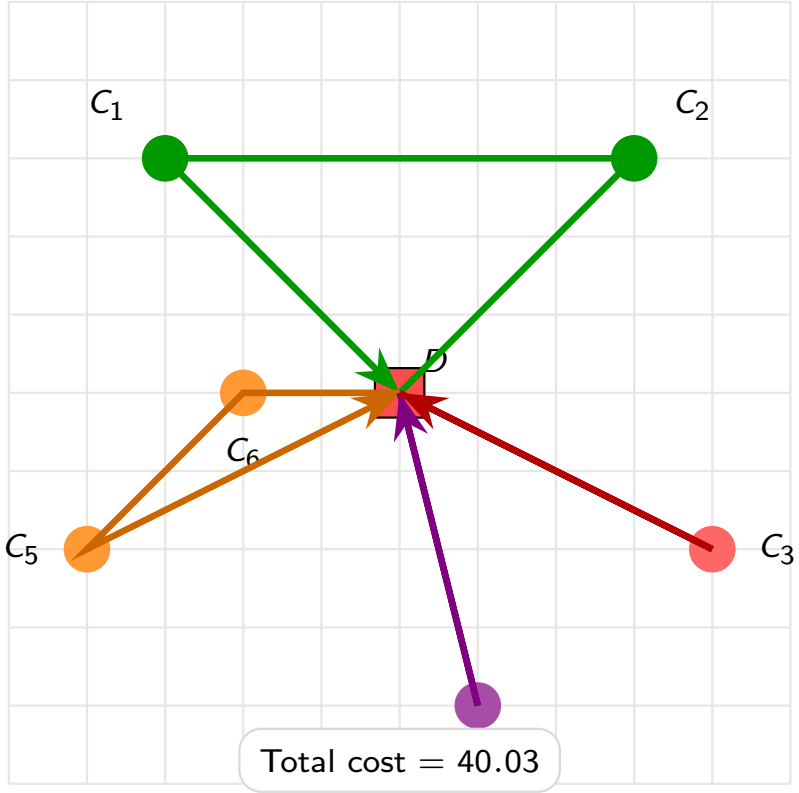
1. Start at $D(5,5)$
2. Nearest unvisited: $C_2(8,8)$, $d$=4.24
3. Nearest remaining: $C_1(2,8)$, $d$=6.00
4. Return $D$: $d$=4.24

Route: $D \to C_2 \to C_1 \to D$, cost = **14.48**

**R2** cluster $\{C_6, C_5\}$:

1. Start at $D(5,5)$
2. Nearest: $C_6(3,5)$, $d$=2.00
3. Nearest remaining: $C_5(1,3)$, $d$=2.83
4. Return $D$: $d$=4.47

Route: $D \to C_6 \to C_5 \to D$, cost = **9.30**
R3: $D \to C_4 \to D$, cost = 8.25 R4: $D \to C_3 \to D$, cost = 8.00



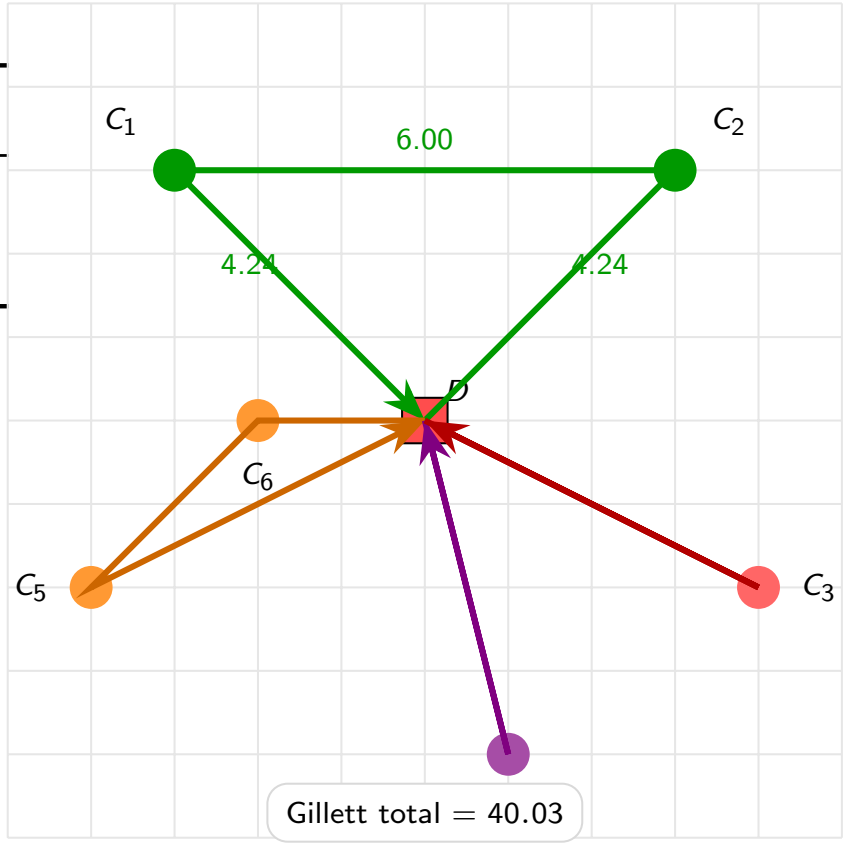
# Gillett Sweep — Step 4: 2-opt Check on Route 1

Route 1 after NN: $D \to C_2 \to C_1 \to D$

Only one interior edge pair to check:

| Edges removed | Edges added | $\Delta$ |
|---|---|---|
| $(D, C_2), (C_1, D)$ | $(D, C_1), (C_2, D)$ | |
| $4.24 + 4.24 = 8.48$ | $4.24 + 4.24 = 8.48$ | 0 |

$\Delta = 0 < \epsilon \Rightarrow$ **No improvement.**
Route 1 is already 2-opt optimal (symmetric distances: order of $C_1, C_2$ does not matter when both are equidistant from depot).




## Final Gillett solution

R1: $D \to C_2 \to C_1 \to D$, cost $= 14.48$
R2: $D \to C_6 \to C_5 \to D$, cost $= 9.30$
R3: $D \to C_4 \to D$, cost $= 8.25$
R4: $D \to C_3 \to D$, cost $= 8.00$
**Total** $= 40.03$

# Data Structures — Node Class

## Node

```python
class Node:
    __slots__ = (
        "id", "x", "y",
        "demand", "angle")

    def __init__(self, id_=0,
                 x=0.0, y=0.0,
                 demand=0):
        self.id     = id_
        self.x      = x
        self.y      = y
        self.demand = demand
        self.angle  = 0.0
```

**Field description:**

| Field | Purpose |
|---|---|
| `id` | Node index (1 = depot) |
| `x, y` | Euclidean coordinates |
| `demand` | Delivery demand $d_i$ |
| `angle` | Polar angle $\theta_i$ |

`__slots__` avoids a per-object dictionary, reducing memory usage for large instances.

# Geometry Helpers

## Euclidean Distance & Polar Angle

```python
def euclidean(a: Node,
              b: Node) -> float:
    return math.hypot(
        a.x - b.x,
        a.y - b.y)

def polar_angle(depot: Node,
                p: Node) -> float:
    a = math.atan2(
        p.y - depot.y,
        p.x - depot.x)
    return (a + 2*math.pi
            if a < 0 else a)
```

**Mathematical definitions:**

Euclidean distance:

$$d(a, b) = \sqrt{(x_a - x_b)^2 + (y_a - y_b)^2}$$

Polar angle (mapped to $[0, 2\pi)$):

$$\theta_i = \text{atan2}(y_i - y_0,\ x_i - x_0)$$

`math.hypot` is numerically stable and avoids intermediate overflow for large coordinates.

# Nearest-Neighbour Route Construction

**nearest_neighbour(cluster, nodes)**

```python
def nearest_neighbour(
        cluster: list,
        nodes: list) -> list:
    remaining = list(cluster)
    route = [1]          # start at depot
    cur = 1
    while remaining:
        best_d = 1e18
        best_v = best_i = -1
        for i, v in enumerate(
                remaining):
            d = euclidean(
                nodes[cur], nodes[v])
            if d < best_d:
                best_d = d
                best_v = v
                best_i = i
        route.append(best_v)
        cur = best_v
        remaining.pop(best_i)
    route.append(1)      # return depot
    return route
```

**Algorithm:**

1. Start at depot (node 1)
2. Greedily visit the nearest unvisited customer
3. Repeat until all customers in the cluster are visited
4. Return to the depot

**Complexity:** $O(|C|^2)$ per cluster $C$

**Output format:**
$[1, c_1, c_2, \ldots, c_k, 1]$
(depot at both ends)

# 2-opt Improvement

**two_opt(route, nodes)**

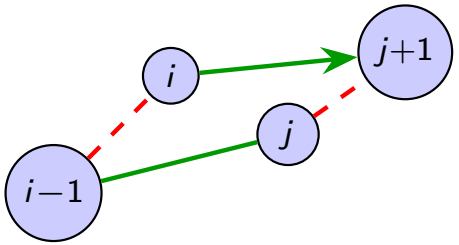
```python
def two_opt(route, nodes):
    n = len(route)
    improved = True
    while improved:
        improved = False
        for i in range(1, n-2):
          for j in range(i+1, n-1):
            before = (
              euclidean(nodes[route[i-1]],
                        nodes[route[i]])
            + euclidean(nodes[route[j]],
                        nodes[route[j+1]]))
            after = (
              euclidean(nodes[route[i-1]],
                        nodes[route[j]])
            + euclidean(nodes[route[i]],
                        nodes[route[j+1]]))
            if after < before - 1e-9:
              route[i:j+1] = \
                route[i:j+1][::-1]
              improved = True
    return route
```

**Idea:** Remove two edges $(i-1, i)$ and $(j, j+1)$; reconnect as $(i-1, j)$ and $(i, j+1)$.

**Improvement condition:**

$$\underbrace{d_{i-1,i} + d_{j,j+1}}_{\text{before}} > \underbrace{d_{i-1,j} + d_{i,j+1}}_{\text{after}}$$

**Complexity:** $O(n^2)$ per pass; repeats until no improvement.




Red = removed, Green = added.

# Sweep Clustering & Route Building

**solve(nodes, dimension, capacity)**

```python
# Compute polar angles
customers = []
for i in range(2, dimension+1):
    nodes[i].angle = polar_angle(
        depot, nodes[i])
    customers.append(nodes[i])

# Sort by angle
customers.sort(
    key=lambda c: c.angle)

# Greedy capacity clustering
clusters, cur, load = [], [], 0
for c in customers:
    if load + c.demand <= capacity:
        cur.append(c.id)
        load += c.demand
    else:
        if cur: clusters.append(cur)
        cur, load = [c.id], c.demand
if cur: clusters.append(cur)

# Build and improve routes
routes = []
for cl in clusters:
    r = nearest_neighbour(cl, nodes)
    r = two_opt(r, nodes)
    routes.append(r)
```

**Clustering rule:**
Add customer $c$ to current route if:

$$\text{load} + d_c \leq Q$$

Otherwise, close the current route and open a new one.

**Pipeline per cluster:**

$$\text{cluster} \xrightarrow{\text{NN}} \text{route} \xrightarrow{\text{2-opt}} \text{improved route}$$

**Total complexity:**
$O(n \log n)$ for sorting
$O(n^2)$ dominated by 2-opt

# Enhanced Sweep — Motivation & Design

## Why "Enhanced"?

The classic Gillett–Miller sweep is fast but sensitive to the starting angle. A single sweep may miss good clusterings. The enhanced solver addresses this with multi-start and richer local search.

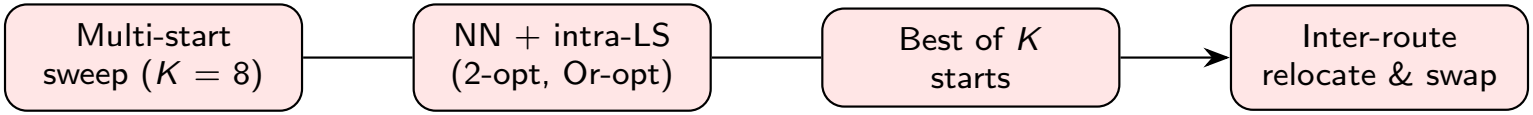
**Gillett–Miller (baseline):**

1. 1 starting angle (0)
2. NN + 2-opt per cluster
3. No inter-route improvement

**Enhanced sweep (this work):**

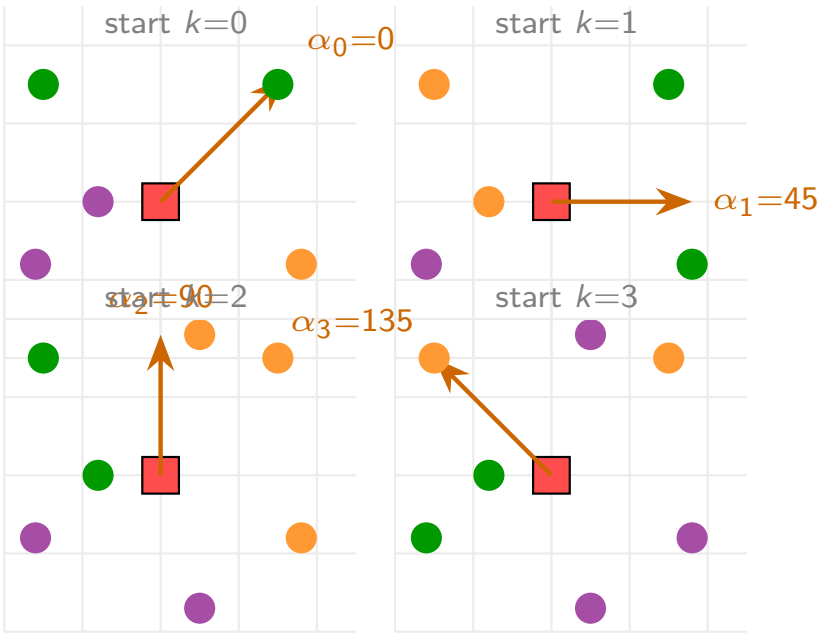
1. $K = 8$ starting angles $(0, \frac{2\pi}{8}, \ldots)$
2. NN + 2-opt + Or-opt (1/2/3) per cluster
3. Inter-route **relocate** (up to 20 passes)
4. Inter-route **swap** (up to 20 passes)

**Enhancement pipeline:**



# Enhanced Sweep — Multi-Start: Different Clusterings

Different sweep origins re-order customers, changing which clusters form.




Each panel shows 4 of the 8 starts. Colours represent different vehicle assignments.

After all $K = 8$ starts, the solver keeps the clustering with the **lowest intra-route cost** and proceeds to inter-route improvement.

## Why multi-start matters

A bad sweep angle can put distant customers in the same vehicle. Trying 8 angles greatly reduces this risk on real instances.

# Enhanced Sweep — Or-opt(1) on Route 2

Route 2 after NN: $D \to C_6 \to C_5 \to D$

**Or-opt(1):** try relocating $C_6$ to after $C_5$:

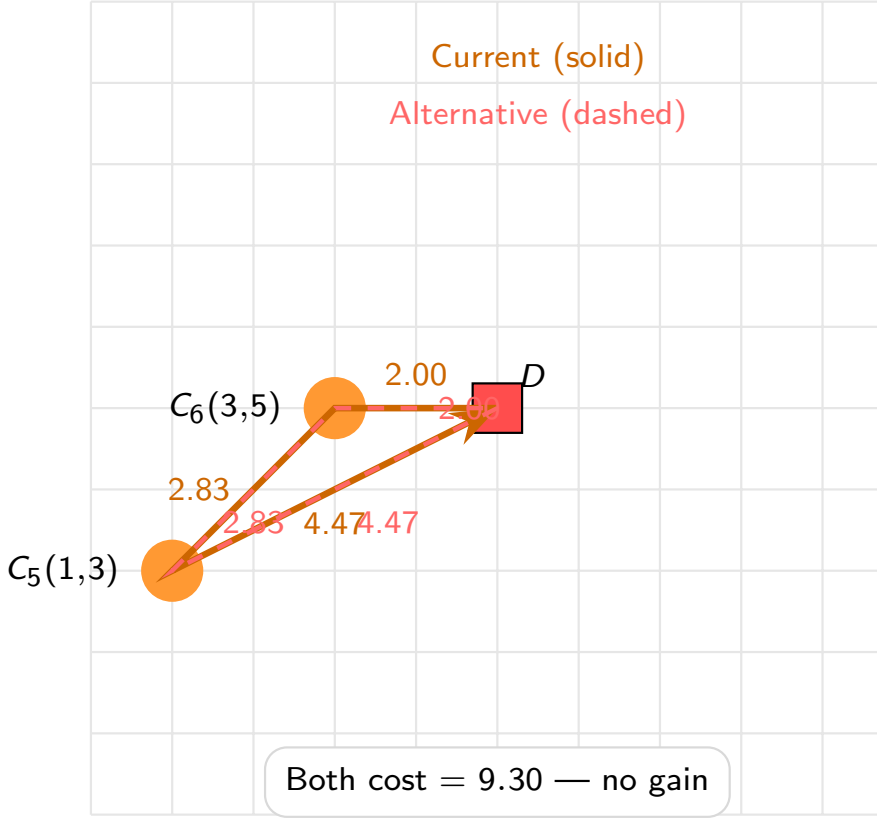
$$g_{\text{rem}} = d(D, C_6) + d(C_6, C_5) - d(D, C_5)$$
$$= 2.00 + 2.83 - 4.47 = 0.36$$

$$g_{\text{ins}} = d(C_5, C_6) + d(C_6, D) - d(C_5, D)$$
$$= 2.83 + 2.00 - 4.47 = 0.36$$

Net gain $= 0.36 - 0.36 = 0 < \epsilon \Rightarrow$ **No move.**

Route 2 is already Or-opt(1) optimal.

The same holds for Or-opt(2) and Or-opt(3) since the route has only 2 customers.



# Enhanced Sweep — Inter-Route Relocate Check

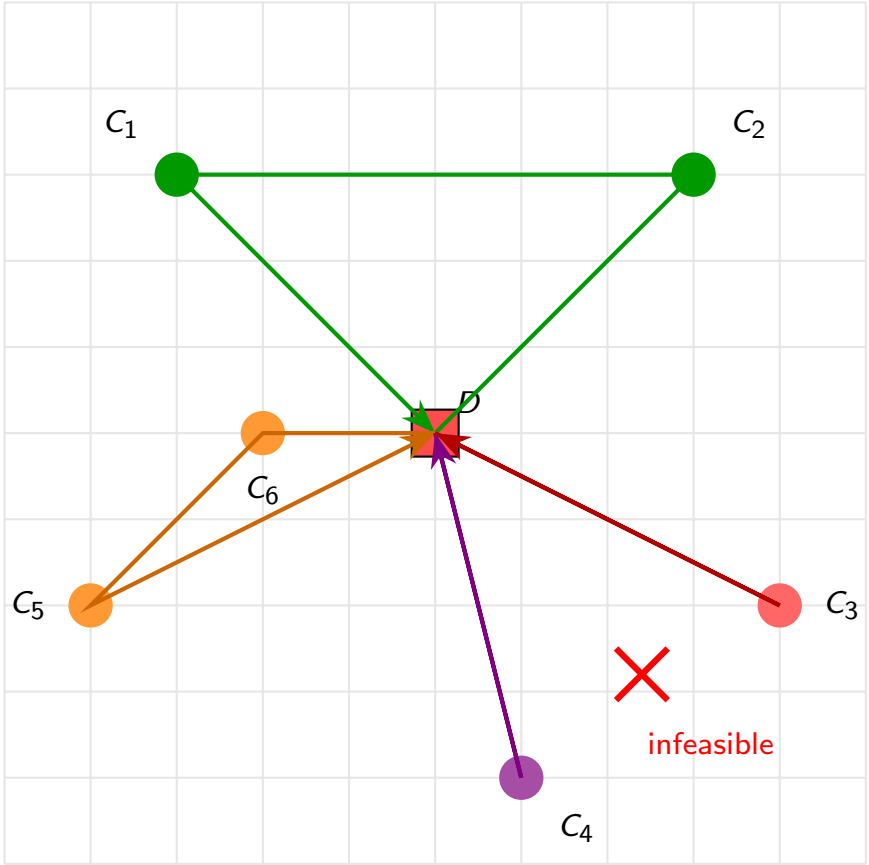
R3: $D \to C_4 \to D$, load $= 18$
R4: $D \to C_3 \to D$, load $= 15$

**Try:** Move $C_4$ from R3 into R4.
Capacity: $15 + 18 = 33 > 30$ — **Infeasible.**

**Try:** Move $C_3$ from R4 into R3.
Capacity: $18 + 15 = 33 > 30$ — **Infeasible.**

**Try:** Move $C_3$ into R1 (after $C_2$).
R1 load: $22 + 15 = 37 > 30$ — **Infeasible.**

**Try:** Move $C_3$ into R2 (after $C_6$).
R2 load: $22 + 15 = 37 > 30$ — **Infeasible.**




## Conclusion

No feasible relocate improves the solution. The 4-route result is confirmed by the enhanced solver too.

# Enhanced Sweep — Inter-Route Swap Check

**Try:** Swap $C_1$ (R1) $\leftrightarrow$ $C_6$ (R2).

Feasibility R1: $22 - 10 + 14 = 26 \leq 30$ ✓

Feasibility R2: $22 - 14 + 10 = 18 \leq 30$ ✓

$\Delta$ cost:

Old: $d(C_2, C_1) + d(C_1, D) + d(D, C_6) + d(C_6, C_5)$

$$= 6.00 + 4.24 + 2.00 + 2.83 = 15.07$$

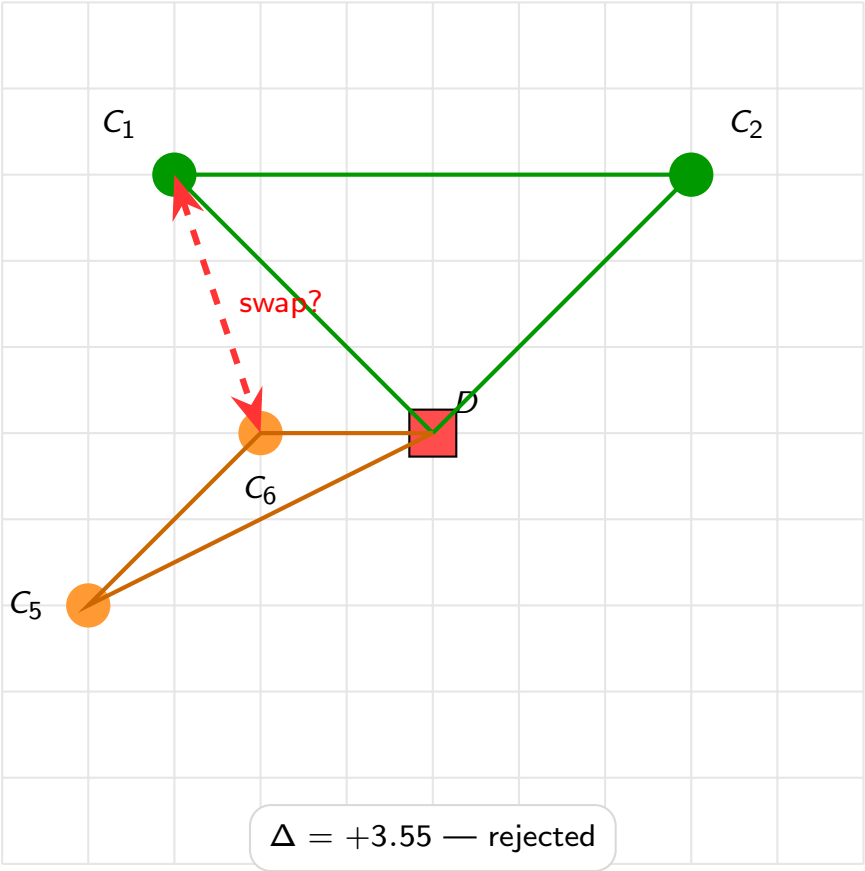
New: $d(C_2, C_6) + d(C_6, D) + d(D, C_1) + d(C_1, C_5)$

$$= 5.10 + 2.00 + 4.24 + 7.28 = 18.62$$

$$\Delta = 18.62 - 15.07 = +3.55 > 0 \Rightarrow$$

**Rejected.**

All other swap pairs yield $\Delta \geq 0$ — no improvement.



# Or-opt: Moving Chains of Customers

**or_opt(route, nodes, length)**

```python
def or_opt(route, nodes, length):
    imp = True
    while imp:
        imp = False
        n = len(route)
        for i in range(1, n-length):
            e = i + length - 1
            rem_gain = (
              dist(nodes[route[i-1]],
                   nodes[route[i]])
            + dist(nodes[route[e]],
                   nodes[route[e+1]])
            - dist(nodes[route[i-1]],
                   nodes[route[e+1]]))
            for j in range(n-1):
                # try forward & reversed
                # insertion after j
                # if gain > 1e-9: move
                ...
    return route
```

**Concept:** Remove a chain of `length` consecutive customers and re-insert it at the best position (forward or reversed).

**Gain calculation:**

$$g_{\text{rem}} = d_{i-1,i} + d_{e,e+1} - d_{i-1,e+1}$$

$$g_{\text{ins}} = d_{j,i} + d_{e,j+1} - d_{j,j+1}$$

Move if $g_{\text{rem}} - g_{\text{ins}} > \epsilon$.

**Called with lengths 1, 2, 3:**

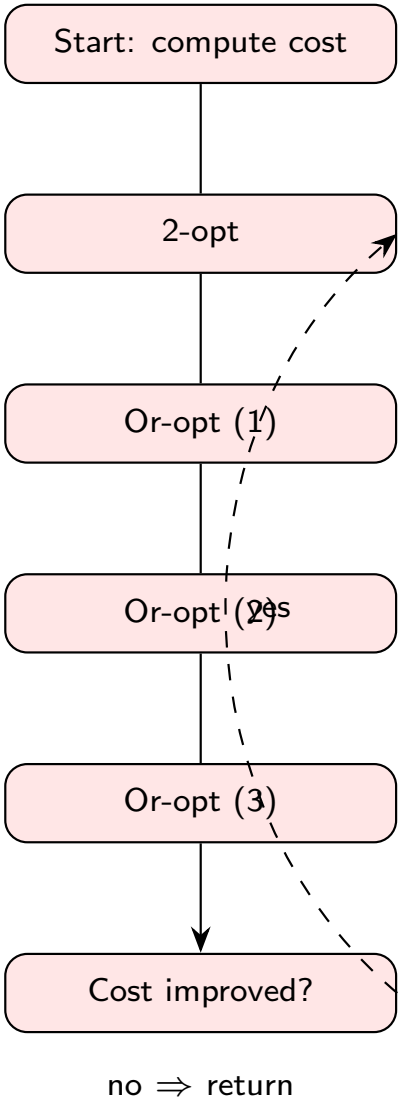
`or_opt(r, nodes, 1)` — relocate
`or_opt(r, nodes, 2)` — move pair
`or_opt(r, nodes, 3)` — move triple

# Intra-Route Local Search Pipeline

## intra_ls(route, nodes)

```
def intra_ls(route, nodes):
    prev = float("inf")
    while True:
        prev = route_cost(route,
                          nodes)
        route = two_opt(route,
                        nodes)
        route = or_opt(route,
                       nodes, 1)
        route = or_opt(route,
                       nodes, 2)
        route = or_opt(route,
                       nodes, 3)
        if route_cost(route, nodes)\
                >= prev - 1e-9:
            break
    return route
```

**Iterates until stable:**



# Inter-Route Relocate

**inter_relocate(routes, nodes, cap)**

```
for a in range(R):
  pi = 1
  while pi + 1 < len(routes[a]):
    cu = routes[a][pi]
    rem_gain = (
      dist(nodes[routes[a][pi-1]],
           nodes[cu])
    + dist(nodes[cu],
           nodes[routes[a][pi+1]])
    - dist(nodes[routes[a][pi-1]],
           nodes[routes[a][pi+1]]))
    best_gain = 1e-9
    for b in range(R):
      if b == a: continue
      if (route_demand(routes[b])
          + nodes[cu].demand > cap):
        continue
      # find best insertion pos
      ...
    if best_b != -1:
      routes[a].pop(pi)
      routes[best_b].insert(
          best_pos, cu)
```

**Goal:** Move a single customer $c_u$ from route $a$ to the best feasible position in any other route $b$.

**Feasibility check:**

$$\sum_{i \in b} d_i + d_{c_u} \leq Q$$

**Net gain:**

$$\Delta = g_{\text{remove}} - g_{\text{insert}}$$

Move if $\Delta > 10^{-9}$.

Empty routes (depot $\rightarrow$ depot only) are automatically pruned after each pass.

# Inter-Route Swap

**inter_swap(routes, nodes, cap)**

```
for a in range(R):
  for pi in range(1,
        len(routes[a])-1):
    for b in range(a+1, R):
      for qi in range(1,
            len(routes[b])-1):
        cu = routes[a][pi]
        cv = routes[b][qi]
        # check both capacity
        # constraints
        ...
        delta = (
          dist(...cv...) +
          dist(...cv...) +
          dist(...cu...) +
          dist(...cu...)
          - dist(...cu...) * 2
          - dist(...cv...) * 2)
        if delta < -1e-9:
          routes[a][pi] = cv
          routes[b][qi] = cu
```

**Goal:** Swap customer $c_u \in$ route $a$ with customer $c_v \in$ route $b$.

**Feasibility (both routes):**

$$\Sigma_a - d_{c_u} + d_{c_v} \leq Q$$

$$\Sigma_b - d_{c_v} + d_{c_u} \leq Q$$

$\Delta$ **cost** (sum of 4 new edges minus 4 old edges). Accept if $\Delta < -10^{-9}$.

**Complexity:** $O(R^2 \cdot n^2)$ per pass; at most 20 passes.

# Multi-Start Sweep ($K = 8$)

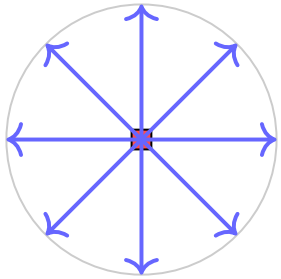
**solve(nodes, dimension, capacity)**

```
K = 8
best_routes = None
best_cost   = float("inf")

for k in range(K):
    start_angle = k*(2*pi/K)

    # rotate customer order
    order = []
    for c in customers:
        a = c.angle - start_angle
        if a < 0: a += 2*pi
        order.append((a, c.id))
    order.sort()

    # cluster + NN + intra-LS
    ...
    c = total_cost(routes, nodes)
    if c < best_cost:
        best_cost   = c
        best_routes = deepcopy(routes)

# Inter-route improvement
for _ in range(20):
    r1 = inter_relocate(...)
    r2 = inter_swap(...)
    if not r1 and not r2: break
    # quick 2-opt + or-opt touchup
```

**Starting angles:**

$$\alpha_k = \frac{2\pi k}{8}, \quad k = 0, 1, \ldots, 7$$

Each start produces a different clustering. The best solution (lowest total cost) is kept.



8 sweep directions

# Worked Example — Final Results

**Instance: 6 customers, $Q = 30$, Depot $(5, 5)$**

| Algorithm | Routes | Total Cost | Note |
|---|---|---|---|
| Gillett & Miller | 4 | 40.03 | Single start |
| Enhanced Sweep | 4 | 40.03 | Best of 8 starts (identical here) |

**Route breakdown:**

- R1: $D \to C_2 \to C_1 \to D$, load 22, cost 14.48
- R2: $D \to C_6 \to C_5 \to D$, load 22, cost 9.30
- R3: $D \to C_4 \to D$, load 18, cost 8.25
- R4: $D \to C_3 \to D$, load 15, cost 8.00

**Key observations:**

- $C_3 + C_4$: $15 + 18 = 33 > 30$ — cannot share a vehicle.
- 4 routes are structurally unavoidable for $Q = 30$.
- Enhanced confirms via relocate/swap that no improvement exists.
- On larger instances the multi-start and inter-route operators give measurable gains.

# Algorithm Comparison: Gillett vs. Enhanced

| Feature | Gillett & Miller | Enhanced Sweep |
|---|---|---|
| Starting angles | 1 (fixed at 0) | 8 (evenly spaced) |
| Intra-route LS | 2-opt only | 2-opt + Or-opt (1/2/3) |
| Inter-route LS | None | Relocate + Swap (20 passes) |
| Post-move touch-up | None | 2-opt + Or-opt (1) |
| Solution quality | Good | Better (typically) |
| Runtime | Very fast | Moderate |
| Output files | `_sweep_gillett.*` | `_enhanced.*` |

**Key Insight**

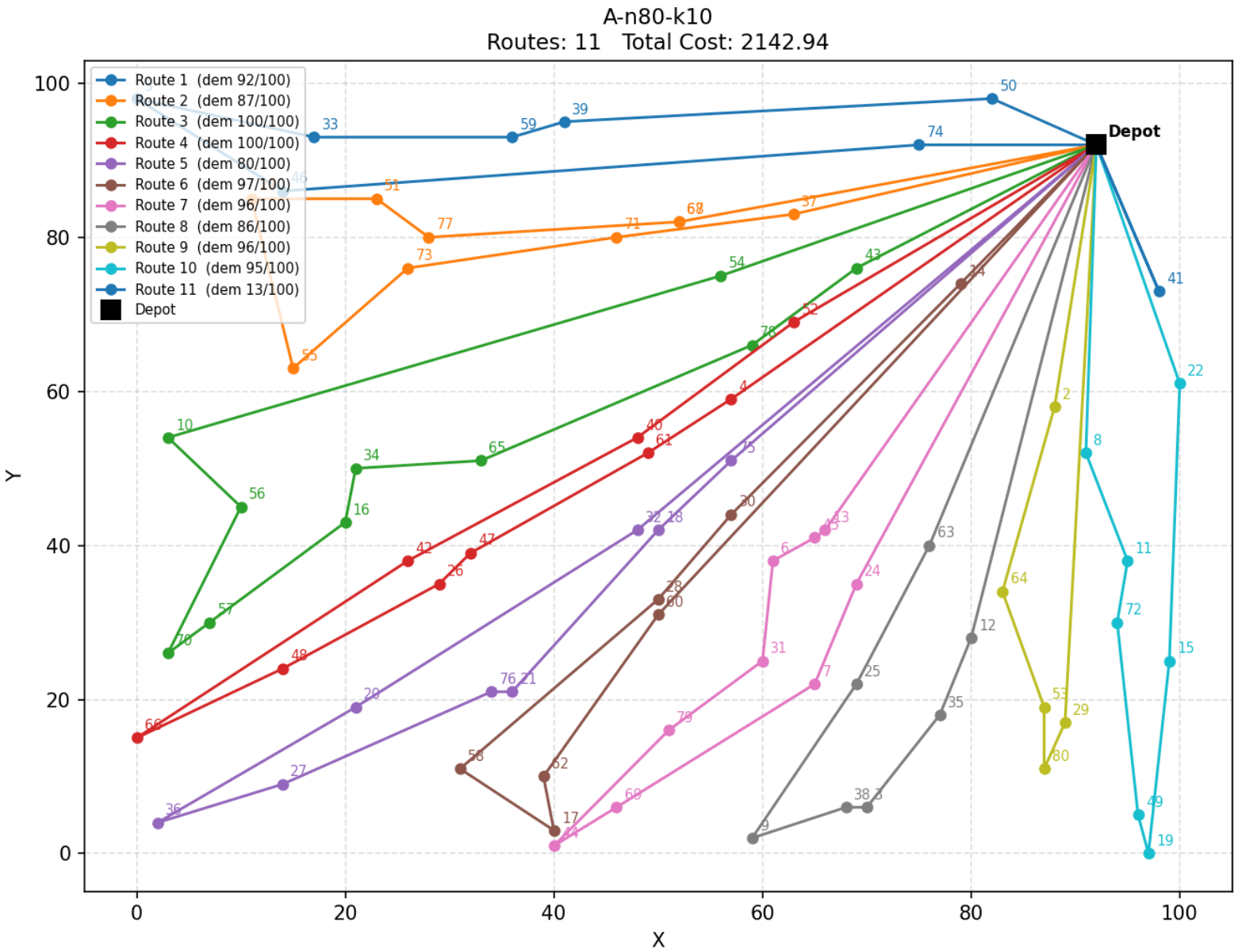
The $K$-start approach hedges against bad sweep angles. The inter-route operators allow customers to "migrate" between routes, fixing imbalances that no intra-route method can resolve.

# Sample Output: comparison_results.txt

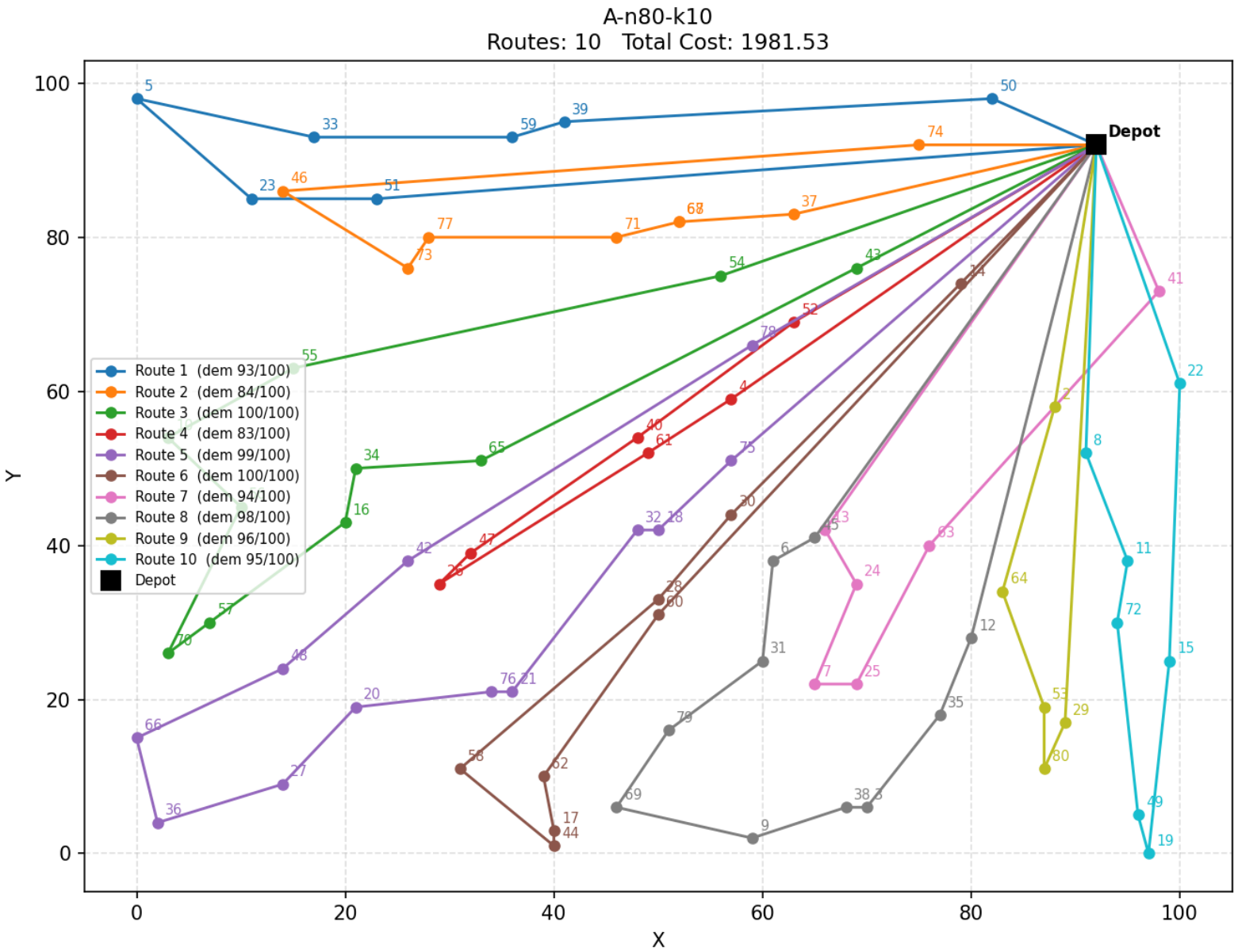
| Instance | Optimal(.sol) | Gillett Sweep | Enhanced | GapGilt% | GapEnh% |
|---|---|---|---|---|---|
| A-n32-k5 | 784.00 | 885.04 | 885.04 | 12.89 | 12.89 |
| A-n33-k5 | 661.00 | 792.40 | 698.30 | 19.88 | 5.64 |
| A-n33-k6 | 742.00 | 875.32 | 751.65 | 17.97 | 1.30 |
| A-n34-k5 | 778.00 | 868.75 | 786.44 | 11.66 | 1.08 |
| A-n36-k5 | 799.00 | 950.36 | 874.86 | 18.94 | 9.49 |
| A-n37-k5 | 669.00 | 841.56 | 737.31 | 25.79 | 10.21 |
| A-n37-k6 | 949.00 | 1136.51 | 1038.62 | 19.76 | 9.44 |
| A-n38-k5 | 730.00 | 880.83 | 776.05 | 20.66 | 6.31 |
| A-n39-k5 | 822.00 | 1014.97 | 881.35 | 23.48 | 7.22 |
| A-n39-k6 | 831.00 | 991.54 | 896.92 | 19.32 | 7.93 |
| A-n44-k6 | 937.00 | 1166.04 | 1055.96 | 24.44 | 12.70 |
| A-n45-k6 | 944.00 | 1116.09 | 1049.70 | 18.23 | 11.20 |
| A-n45-k7 | 1146.00 | 1342.89 | 1306.53 | 17.18 | 14.01 |
| A-n46-k7 | 914.00 | 1039.92 | 961.04 | 13.78 | 5.15 |
| A-n48-k7 | 1073.00 | 1155.59 | 1147.35 | 7.70 | 6.93 |
| A-n53-k7 | 1010.00 | 1176.36 | 1088.12 | 16.47 | 7.73 |
| A-n54-k7 | 1167.00 | 1368.72 | 1320.84 | 17.29 | 13.18 |
| A-n55-k9 | 1073.00 | 1204.19 | 1172.99 | 12.23 | 9.32 |
| A-n60-k9 | 1354.00 | 1680.92 | 1450.92 | 24.14 | 7.16 |
| A-n61-k9 | 1034.00 | 1226.47 | 1093.82 | 18.61 | 5.79 |
| A-n62-k8 | 1288.00 | 1624.06 | 1386.20 | 26.09 | 7.62 |
| A-n63-k10 | 1314.00 | 1555.07 | 1409.67 | 18.35 | 7.28 |
| A-n63-k9 | 1616.00 | 1828.05 | 1789.02 | 13.12 | 10.71 |
| A-n64-k9 | 1401.00 | 1599.79 | 1508.46 | 14.19 | 7.67 |
| A-n65-k9 | 1174.00 | 1373.00 | 1286.49 | 16.95 | 9.58 |
| A-n69-k9 | 1159.00 | 1271.82 | 1255.55 | 9.73 | 8.33 |
| A-n80-k10 | 1763.00 | 2142.94 | 1981.53 | 21.55 | 12.40 |
| AVERAGE | 1041.93 | 1226.27 | 1132.99 | 17.69 | 8.74 |

Enhanced wins: 26 instance(s) Gillett wins: 0 Tied: 1 Total: 27

# Route Plot: A-n80-k10 — Gillett Sweep



# Route Plot: A-n80-k10 — Enhanced Sweep



# Complexity Summary

| Operation | Complexity | Used in | Notes |
|---|---|---|---|
| Polar angle sort | $O(n \log n)$ | Both solvers | Once per start |
| Greedy clustering | $O(n)$ | Both solvers | Linear scan |
| Nearest-neighbour | $O(\|C\|^2)$ per cluster | Both solvers | $\sum \|C\| = n$ |
| 2-opt | $O(n^2)$ per pass | Both solvers | Repeats until stable |
| Or-opt (len $\ell$) | $O(n^2)$ per pass | Enhanced | $\ell \in \{1, 2, 3\}$ |
| Inter-route relocate | $O(R \cdot n^2)$ per pass | Enhanced | Up to 20 passes |
| Inter-route swap | $O(R^2 \cdot n^2)$ per pass | Enhanced | Up to 20 passes |
| Multi-start | $\times K = 8$ | Enhanced | Best of 8 kept |

$n$ = number of customers, $R$ = number of routes, $C$ = cluster

# Genetic Algorithm – Background

Genetic Algorithms (GAs) are population-based evolutionary search methods inspired by natural selection [11, 12].

**Why GA for CVRP?**

- Can escape local optima via population diversity
- Flexible representation for combinatorial problems
- Easily extended with domain-specific operators
- No gradient information required

**Giant-tour representation**

- Chromosome = permutation of all $n$ customer IDs
- Decoded by greedy split: scan the tour and start a new route whenever the next customer would exceed $C$
- Simple, always feasible, no repair needed

# Original GA – Algorithm

**Algorithm 1** Original Genetic Algorithm for CVRP

```
P ← RandomPopulation(N_pop)
for g = 1 to G_max do
    Evaluate fitness: f(c) = TotalDist(Decode(c)) for all c ∈ P
    P' ← Elites(P, e)                                  ▷ keep top e unchanged
    while |P'| < N_pop do
        p_1, p_2 ← TournamentSelect(P, k = 3)
        c_1, c_2 ← OX1Crossover(p_1, p_2) with prob. p_c
        c_1 ← Mutate(c_1);  c_2 ← Mutate(c_2)
        Append c_1, c_2 to P'
    end while
    P ← P'
end for
return best chromosome seen
```

# Original GA – Operators

**Order Crossover (OX1)** [13]

1. Choose a random segment $[a, b]$ from parent $P_1$
2. Copy $P_1[a \dots b]$ into the child at positions $a \dots b$
3. Fill remaining positions from $P_2$ in order, wrapping around

Preserves the relative ordering of customer visits.

**Mutation (50/50 choice)**

- *Swap*: exchange two random positions
- *Inversion*: reverse a random sub-segment

**Parameters used**

| Parameter | Value |
|---|---|
| Population size | 100 |
| Generations | 300 |
| Crossover rate | 0.9 |
| Mutation rate | 0.2 |
| Tournament size | 3 |
| Elite size | 5 |
| Random seed | 42 |

# Proposed Modifications – Overview

Three targeted improvements are proposed to address the main weaknesses of the original GA:

1. **Nearest-Neighbour Seeded Initialisation**
   *Problem solved*: random initialisation produces very poor starting tours; the GA wastes generations escaping terrible solutions.

2. **Intra-route 2-opt Local Search**
   *Problem solved*: crossover and mutation alone cannot reliably eliminate edge crossings inside a single route; local search is essential to reach high-quality routes.

3. **Adaptive Mutation Rate**
   *Problem solved*: a fixed mutation rate is either too low (population stagnates in a local optimum) or too high (good solutions are destroyed).

# Modification 1 – Nearest-Neighbour Seeded Initialisation

**Idea**

Seed 20% of the initial population with greedy nearest-neighbour tours, each starting from a different customer. The remaining 80% stay random.

**Nearest-neighbour heuristic**:

1. Start at a given customer
2. Repeatedly move to the closest unvisited customer
3. Stop when all customers are added

**Reasoning**:

- NN tours are typically 20–30% above optimal, far better than random
- Provides the GA with diverse but high-quality starting seeds
- Retaining 80% random individuals preserves population diversity
- Particularly impactful on large instances (Golden, 200–483 nodes) where random initialisation converges extremely slowly

# Modification 2 – Intra-route 2-opt Local Search

**Idea**

Every $L$ generations, apply a 2-opt improvement pass to each route of every newly created offspring before inserting it into the population.

**2-opt**: repeatedly reverse a sub-segment $[i, j]$ of a route if doing so reduces its travel distance [14]:

$$\text{remove edges } (v_{i-1}, v_i) \text{ and } (v_j, v_{j+1}); \quad \text{add } (v_{i-1}, v_j) \text{ and } (v_i, v_{j+1})$$

**Reasoning**:

- OX1 crossover preserves *global* customer order but disrupts *local* route geometry, creating crossings
- 2-opt directly eliminates these crossings within each route
- Applied periodically (every $L = 10$ gen) – not every generation – to bound the extra computation cost
- Routes longer than 40 nodes skip 2-opt to prevent $O(n^2)$ blow-up

# Modification 3 – Adaptive Mutation Rate

**Idea**

Monitor stagnation (generations without improvement). When stagnation exceeds a threshold, gradually raise the mutation rate to inject diversity. Reset to the base rate upon finding a new best solution.

**Rule**:

$$\mu = \begin{cases} \mu_{\text{base}} & \text{if stagnation} < S_{\text{limit}} \\ \min\left(0.8,\ \mu_{\text{base}} \times \left(1 + \frac{\text{stagnation}}{S_{\text{limit}}}\right)\right) & \text{otherwise} \end{cases}$$

**Parameters**: $\mu_{\text{base}} = 0.2,\ \ S_{\text{limit}} = 25.$

**Reasoning**:

- A fixed low rate is insufficient to escape plateaus; a fixed high rate randomly destroys good building blocks
- Adaptive rate raises diversity only when the population is stuck, then returns to a conservative rate once improvement resumes

# Experimental Setup

**GA Parameters**

| Parameter | Value |
|---|---|
| Population size | 100 |
| Generations | 300 |
| Crossover rate | 0.9 |
| Base mutation rate | 0.2 |
| Tournament size | 3 |
| Elite size | 5 |
| 2-opt interval $L$ | 10 |
| Stagnation limit | 25 |
| Random seed | 42 |

**Evaluation metric**

$$\text{Gap\%} = \frac{\text{found} - \text{known best}}{\text{known best}} \times 100$$

- Lower gap = better solution
- Gap = 0% means optimal solution found
- Both algorithms run from the same seed for fair comparison
- 60 instances total across 3 datasets

# Results – Dataset A (Augerat, 27 instances)

| Instance | Optimal | Orig GA | Gap% | Impr GA | Gap% |
|---|---|---|---|---|---|
| A-n32-k5 | 784 | 924 | 17.86 | 834 | 6.38 |
| A-n33-k5 | 661 | 736 | 11.35 | 665 | 0.61 |
| A-n38-k5 | 730 | 902 | 23.56 | 738 | 1.10 |
| A-n45-k6 | 944 | 1292 | 36.86 | 1010 | 7.01 |
| A-n53-k7 | 1010 | 1556 | 54.06 | 1093 | 8.22 |
| A-n63-k10 | 1314 | 1841 | 40.11 | 1345 | 2.36 |
| A-n80-k10 | 1763 | 2895 | 64.21 | 1933 | 9.64 |
| **Avg (27)** | | | **+36.98** | | **+6.66** |

# Results – Dataset E (Eilon, 13 instances)

| Instance | Optimal | Orig GA | Gap% | Impr GA | Gap% |
|---|---|---|---|---|---|
| E-n13-k4 | 247 | 247 | 0.00 | 247 | 0.00 |
| E-n23-k3 | 569 | 570 | 0.18 | 569 | 0.00 |
| E-n30-k3 | 534 | 584 | 9.36 | 541 | 1.31 |
| E-n51-k5 | 521 | 809 | 55.28 | 551 | 5.76 |
| E-n76-k7 | 682 | 1213 | 77.86 | 786 | 15.25 |
| E-n101-k8 | 815 | 1650 | 102.45 | 928 | 13.87 |
| E-n101-k14 | 1067 | 1984 | 85.94 | 1256 | 17.71 |
| **Avg (13)** | | | **+47.33** | | **+10.37** |

# Results – Dataset Golden (20 large instances)

| Instance | Optimal | Orig GA | Gap% | Impr GA | Gap% |
|---|---|---|---|---|---|
| Golden_1 | 5623.47 | 22239 | 295.45 | 5809 | 3.30 |
| Golden_10 | 735.43 | 3443 | 368.17 | 752 | 2.25 |
| Golden_11 | 911.98 | 4847 | 431.47 | 908 | −0.44 |
| Golden_12 | 1100.67 | 6831 | 520.63 | 1118 | 1.57 |
| Golden_13 | 857.19 | 3054 | 256.31 | 955 | 11.41 |
| Golden_17 | 707.76 | 2323 | 228.24 | 822 | 16.14 |
| Golden_9 | 579.70 | 2228 | 284.35 | 554 | −4.43 |
| **Avg (20)** | | | **+373.22** | | **+11.55** |

*Negative gaps: Improved GA found solutions better than the listed known best.*

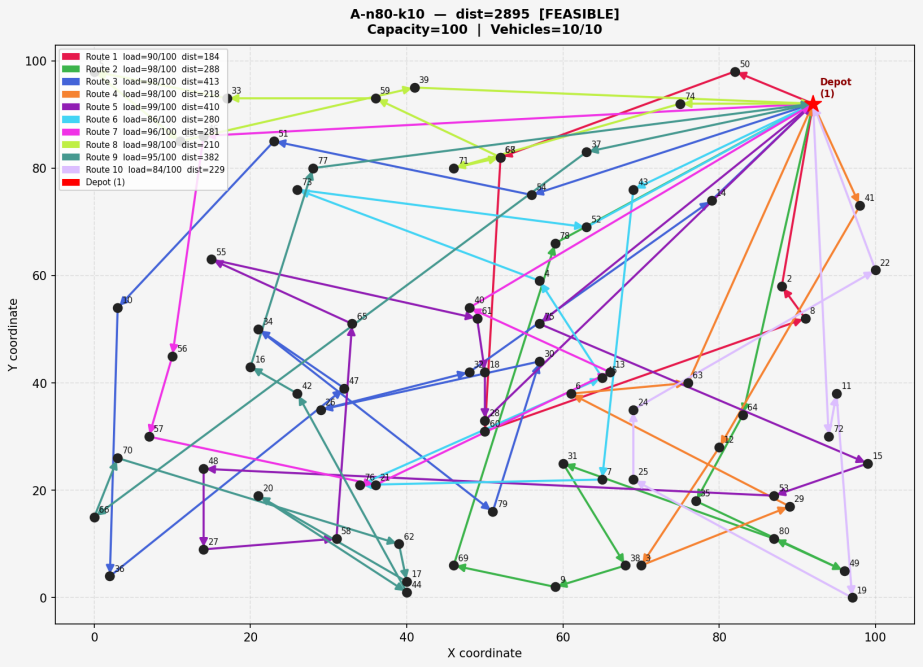
# Summary of Results

| Dataset | Avg Gap% ↓ | | Avg Time (s) | |
|---|---|---|---|---|
| | Orig. GA | Imp. GA | Orig. GA | Imp. GA |
| A (27 inst.) | 36.98 | 6.66 | 0.5 | 1.2 |
| E (13 inst.) | 47.33 | 10.37 | 0.8 | 4.0 |
| Golden (20 inst.) | 373.22 | 11.55 | 3.0 | 98.0 |
| **Overall (60)** | **151.30** | **9.09** | | |

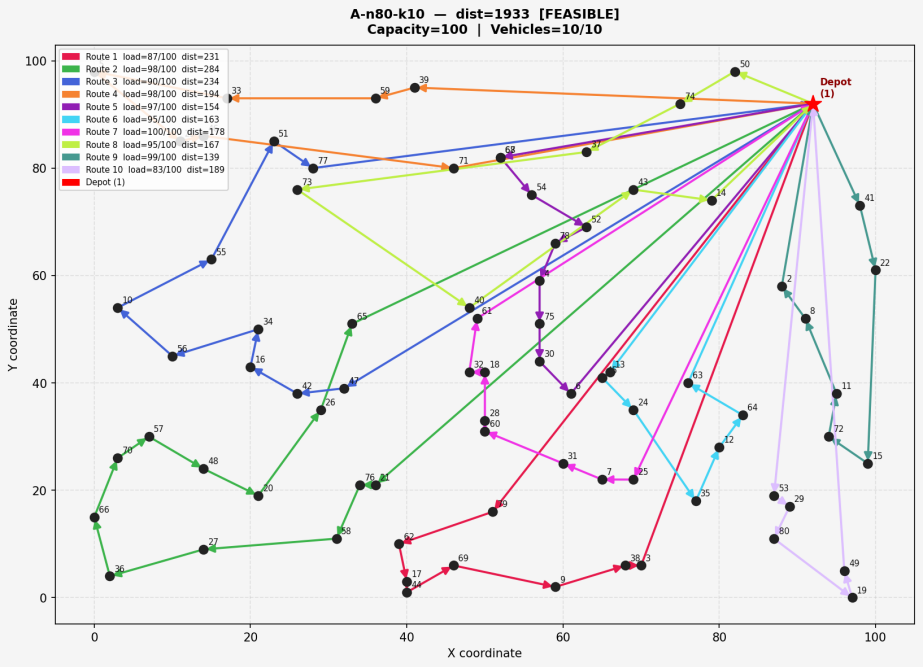
- Improved GA is **16.6**$\times$ **closer to optimal** on average
- Largest gain on Golden (large instances): random init completely fails; NN + 2-opt brings gap from $>$370% down to $\sim$11%
- Small E instances ($n < 30$): both algorithms find near-optimal
- Trade-off: Improved GA is $\sim$2.5$\times$ slower on average

# Route Visualisation – Example

**Original GA** – A-n80-k10 (cost = 2895)




**Improved GA** – A-n80-k10 (cost = 1933)




*Optimal = 1763. Improved GA gap: 9.64% vs Original GA gap: 64.21%. The 2-opt local search visibly eliminates route crossings.*

# Discussion – Impact of Each Modification

| Modification | Main effect | Impact |
|---|---|---|
| NN init | Provides good starting tours; reduces wasted early generations | High |
| 2-opt LS | Eliminates intra-route edge crossings that genetic operators alone cannot remove | Very high |
| Adaptive $\mu$ | Escapes stagnation without permanently sacrificing exploitation quality | Medium |

- The 2-opt local search is the most expensive modification but yields the greatest quality gain – a classic metaheuristic trade-off
- NN initialisation is essentially free; its benefit is largest on the Golden benchmark where random tours are exponentially worse
- All three modifications work synergistically: NN provides a good starting point, 2-opt refines routes, and adaptive mutation explores further when the search plateaus

# Conclusion

## Summary

We implemented and evaluated a standard Genetic Algorithm and an improved variant for the Capacitated Vehicle Routing Problem over 60 benchmark instances from three well-known datasets.

**Key findings:**

- The Original GA finds feasible solutions but struggles with solution quality, especially on large Golden instances (>370% avg gap)
- All three proposed modifications contribute to quality improvement
- The Improved GA achieves an average gap of **9.09%** vs **151.30%** for the Original GA – a 16.6× improvement
- The giant-tour + greedy split decoding always produces capacity-valid solutions with no repair step needed

# References I


[1] Philippe Augerat et al. *Computational results with a branch and cut code for the capacitated vehicle routing problem*. Tech. rep. RR-949-M. Université Joseph-Fourier, Grenoble, France, 1995.

[2] Gilbert Laporte and Yves Nobert. "A branch and bound algorithm for the capacitated vehicle routing problem". In: *OR Spektrum* 5 (1983), pp. 77–85. DOI: 10.1007/BF01720015.

[3] Ricardo Fukasawa et al. "Robust branch-and-cut-and-price for the capacitated vehicle routing problem". In: *Mathematical Programming* 106.3 (2006), pp. 491–511. DOI: 10.1007/s10107-005-0644-x.

[4] Nicos Christofides and Samuel Eilon. "Set M and CMT instances". In: *Summary of the VRPNC Benchmark Instances* (1979).

# References II


[5] Amina Shour, Kassem Danash, and Abbas Tarhini. "Modified Clarke Wright Algorithms for Solving the Realistic Vehicle Routing Problem". In: *2015 Third International Conference on Technological Advances in Electrical, Electronics and Computer Engineering (TAEECE)*. 2015, pp. 89–93. DOI: 10.1109/TAEECE.2015.7113606.

[6] Neha Sehta and Urjita Thakar. "Sweep Nearest Algorithm for Capacitated Vehicle Routing Problem". In: *2021 IEEE 18th India Council International Conference (INDICON)*. 2021, pp. 1–6. DOI: 10.1109/INDICON52576.2021.9691603.

[7] Gulay Barbarosoglu and Demet Ozgur. "A tabu search algorithm for the vehicle routing problem". In: *Computers & Operations Research* 26.3 (1999), pp. 255–270. DOI: 10.1016/S0305-0548(98)00047-1.

# References III


[8] Ali Najm Jasim and Lamia Chaari Fourati. "Guided Genetic Algorithm for Solving Capacitated Vehicle Routing Problem With Unmanned-Aerial-Vehicles". In: *IEEE Access* 12 (2024), pp. 106333–106358. DOI: 10.1109/ACCESS.2024.3438079.

[9] Wenjuan Gu et al. "Adaptive large neighborhood search for the commodity constrained split delivery VRP". In: *Computers & Operations Research* 112 (2019), p. 104761. DOI: 10.1016/j.cor.2019.07.019.

[10] Petr Stodola et al. "Using the Ant Colony Optimization Algorithm for the Capacitated Vehicle Routing Problem". In: *Proceedings of the 16th International Conference on Mechatronics – Mechatronika 2014*. 2014, pp. 503–510. DOI: 10.1109/MECHATRONIKA.2014.7018311.

[11] John H. Holland. "Adaptation in Natural and Artificial Systems". In: (1975).

# References IV


[12] David E. Goldberg. "Genetic Algorithms in Search, Optimization and Machine Learning". In: (1989).

[13] Lawrence Davis. "Applying adaptive algorithms to epistatic domains". In: (1985), pp. 162–164.

[14] Shen Lin. "Computer solutions of the traveling salesman problem". In: *Bell System Technical Journal* 44.10 (1965), pp. 2245–2269. DOI: 10.1002/j.1538-7305.1965.tb04146.x.

## Thank You!

All source code, benchmark datasets, solution outputs, and route plots are available at:

`https://github.com/TH-014/CVRP.git`

The repository includes implementations of the custom Branch-and-Cut solver, the OR-Tools-based solver, Sweep algorithm and Genetic algorithm based solver and their modified versions, shell scripts for batch benchmarking, and all generated plots for the Augerat Set-A instances.